Você é um especialista em arquitetura de sistemas com IA, automação com n8n e construção de MVPs funcionais para times de vendas (RevOps).
Estou desenvolvendo um projeto de Lead Scoring para um challenge técnico, com foco em entregar uma solução funcional (não só teórica).
CONTEXTO DO PROBLEMA:
Tenho um CRM com ~8.800 oportunidades e 35 vendedores. Hoje, a priorização é feita no feeling, o que gera perda de tempo, baixa conversão e desperdício de esforço comercial.
OBJETIVO:
Construir um sistema funcional que:
- Analise oportunidades (deals)
- Priorize automaticamente (score)
- Gere recomendações acionáveis para o vendedor
- Seja utilizável via dashboard ou interface simples
STACK PREFERIDA:
- n8n (orquestração)
- Banco de dados (ex: Supabase/Postgres)
- API de IA (OpenAI ou Claude)
- Dashboard pronto (Metabase, Retool, etc)
DADOS DISPONÍVEIS:
Tenho 4 tabelas:
- accounts (empresa, receita, funcionários, setor)
- products (produto e preço)
- sales_teams (vendedores, manager, região)
- sales_pipeline (oportunidades com stage, datas, valor, etc)
A tabela central é sales_pipeline, conectando todas as outras.

QUERO QUE VOCÊ:
Me ajude a planejar a arquitetura completa da solução, com foco em execução rápida (4–6h), clareza e impacto real.

⚙️ 2. ARQUITETURA DA SOLUÇÃO
📌 Desenho do fluxo
Descreva o fluxo completo do sistema:
- Entrada de dados (de onde vem e como são estruturados)
- Processamento (regras + IA dentro do n8n)
- Output (onde o resultado aparece e como o vendedor usa)
Inclua um fluxo passo a passo (tipo pipeline).

📌 Decisões técnicas
Sugira e justifique:
- Uso do n8n (como organizar workflows)
- Qual banco usar e por quê
- Qual API de IA usar
- Se devo usar dados reais completos ou amostra
- Ferramenta ideal para dashboard (rápida e funcional)

📌 Lógica de negócio
Defina claramente:

- Quais regras determinísticas usar no scoring
- Quais features são mais importantes (baseado nos dados disponíveis)
- Onde entra IA (e exatamente o que ela faz)
- Por que usar IA nessas partes (e não regra fixa)

🤖 3. USO DE IA
📌 Estratégia
- Onde a IA realmente agrega valor?
- Onde NÃO usar IA (e usar regra simples)?

📌 Prompts
Crie:
- Prompt inicial (versão simples)
- Melhorias iterativas
- Prompt final otimizado (pronto pra produção no n8n)

📌 Validação
- Quais erros a IA pode cometer nesse cenário?
- Como posso reduzir inconsistência?
- Estratégias para tornar o output mais confiável

🛠️ 4. IMPLEMENTAÇÃO
📌 Sistema rodando
Explique:
- Como estruturar o workflow no n8n (node por node)
- Como integrar banco + IA
- Como garantir que o fluxo roda de ponta a ponta

📌 Demonstração
- Como simular execução para demo
- Como mostrar valor rapidamente

📌 Extras (se sobrar tempo)
Sugira melhorias de alto impacto como:
- Alertas automáticos (Slack, WhatsApp, etc)
- “Modo foco” com top leads
- Insights automáticos

REGRAS IMPORTANTES:
- Priorize simplicidade e execução rápida
- Evite overengineering
- Pense como alguém construindo um MVP funcional em poucas horas
- Sempre explique o "porquê" das decisões
- Foque em algo que um vendedor realmente usaria

FORMATO DA RESPOSTA:
- Estruturado por seções
- Prático (com exemplos e pseudo-código se necessário)
- Direto ao ponto (sem enrolação)

Mostrar mais
08:59

Sistema de Lead Scoring com IA — Arquitetura Completa para MVP

Vou te entregar um plano de execução real, pensado para rodar em 4–6h com impacto imediato.

⚙️ 2. ARQUITETURA DA SOLUÇÃO

Fluxo Completo (Pipeline)

```
[Supabase/Postgres]
        ↓
  n8n Workflow
        ↓
  ┌──────────────────────────────────────┐
    1. Fetch deals (sales_pipeline)
    2. Enrich com accounts/products
    3. Score determinístico (regras)
    4. Prompt → Claude/OpenAI
    5. Parse output + salvar score
  └──────────────────────────────────────┘
        ↓
  Tabela: scored_leads (Supabase)
        ↓
  [Metabase Dashboard]
        ↓
  Vendedor abre → vê Top 10 prioritários + recomendação
```

Entrada: `sales_pipeline` JOIN `accounts` JOIN `products` JOIN `sales_teams`
Processamento: n8n (regras JS + chamada IA)
Output: Tabela `scored_leads` com `score`, `tier` (A/B/C) e `ai_recommendation`

Decisões Técnicas

n8n Use dois workflows separados:

- `WF-01: Score Batch` — roda de hora em hora, processa todos os deals abertos
- `WF-02: Score On-Demand` — webhook, reprocessa um deal específico via dashboard

Por quê separar? Evita timeout. O batch pode demorar, o on-demand precisa ser rápido.

Banco: Supabase (Postgres)

- Já tem REST API nativa → n8n conecta sem plugin extra
- Dashboard Metabase conecta via Postgres direto
- Grátis no tier inicial, suficiente para 8.800 registros
- Cria uma tabela extra: `scored_leads` sem mexer nas originais

API de IA: Claude (claude-sonnet-4)

- Output mais estruturado que GPT para JSON
- Melhor em seguir instruções de formato — crítico para parsear resposta no n8n

- Se já tem conta Anthropic, use. Se não, OpenAI GPT-4o também funciona

Dados: amostra ou completo?

- Para dev/demo: amostra de 200–500 deals (filtro: `stage NOT IN ('closed_won', 'closed_lost')`)
- Para produção: todos os deals abertos (~estimativa: 20–30% dos 8.800)
- Por quê não tudo? A IA custa por token. Com 500 deals a $0.003/deal = ~$1.50 por rodada

Dashboard: Metabase

- Conecta direto no Postgres em 5 minutos
- Filtros, tabelas e cards sem código
- Alternativa: Retool (mais bonito, mais complexo) — só se sobrar tempo

Lógica de Negócio

Scoring Determinístico (70% do score final)

```
// Node "Code" no n8n — roda para cada deal

function calcScore(deal) {
  let score = 0;

  // 1. VALOR DO DEAL (0-25 pts)
  if (deal.amount > 100000)      score += 25;
  else if (deal.amount > 50000)  score += 18;
  else if (deal.amount > 20000)  score += 12;
  else if (deal.amount > 5000)   score += 6;

  // 2. STAGE DO PIPELINE (0-20 pts)
  const stageScore = {
    'proposal':        8,
    'negotiation':     15,
    'contract_review': 20,
    'demo_done':       12,
    'qualified':       6,
  };
  score += stageScore[deal.stage] || 0;

  // 3. TEMPO NO STAGE ATUAL (0-15 pts — penaliza stagnação)
  const diasNoStage = daysBetween(deal.stage_updated_at, new Date());
  if (diasNoStage <= 7)       score += 15;
  else if (diasNoStage <= 14) score += 10;
  else if (diasNoStage <= 30) score += 5;
  else                        score += 0; // deal parado = baixa
prioridade

  // 4. PERFIL DA EMPRESA (0-15 pts)
  if (deal.account_revenue > 10000000) score += 8;  // empresa grande
  if (deal.employee_count > 200)       score += 4;
  if (deal.sector === 'technology' || deal.sector === 'finance')
score += 3;

  // 5. PROXIMIDADE DO CLOSE DATE (0-15 pts)
  const diasParaClose = daysBetween(new Date(),
deal.expected_close_date);
  if (diasParaClose <= 7)        score += 15;
  else if (diasParaClose <= 14)  score += 10;
  else if (diasParaClose <= 30)  score += 6;
```

```
  else if (diasParaClose < 0)    score -= 10; // atrasado = urgente
mas penalizado

  // 6. HISTÓRICO DE ATIVIDADE (0-10 pts)
  if (deal.last_activity_days <= 3)  score += 10;
  else if (deal.last_activity_days <= 7)  score += 6;
  else if (deal.last_activity_days > 14)  score -= 5; // sem contato
recente

  return Math.max(0, Math.min(100, score)); // clamp 0-100
}
```

Tier baseado no score:

```
A (70-100): Fechar essa semana
B (40-69):  Nutrir e avançar
C (0-39):   Baixa prioridade / revisar
```

🤖 3. USO DE IA

Onde a IA agrega valor

USE IA para:

- Gerar recomendação em linguagem natural ("Ligue hoje, empresa está em fim de trimestre e valor é alto")
- Detectar padrões de risco ("Deal parado há 25 dias com close date em 5 dias — risco alto")
- Personalizar próximo passo baseado em contexto combinado (setor + stage + valor + histórico)

NÃO USE IA para:

- Calcular o score numérico — use regras (auditável, rápido, barato)
- Rankear os deals — faz matematicamente
- Decidir se o deal está aberto ou fechado — dado estruturado

Por quê essa divisão? Regras = velocidade + auditabilidade + custo zero. IA = interpretação de contexto combinado que regra não captura bem. Você explica para o gestor como o score é calculado (regras), e a IA gera o *insight acionável* em cima disso.

Prompts

Versão simples (ponto de partida):

```
Você é um assistente de vendas. Analise esse deal e dê uma
recomendação.
Deal: {json do deal}
```

Versão intermediária:

```
Você é um especialista em vendas B2B.
Dado o deal abaixo, identifique o principal risco e sugira a próxima
ação.

Deal: {json}
Score calculado: {score}/100
Tier: {A/B/C}

Responda em 2 frases.
```

Prompt final otimizado para produção no n8n:

```
Você é um especialista em vendas B2B consultivas. Analise a
oportunidade abaixo e retorne APENAS um JSON válido, sem texto
adicional, sem markdown.

OPORTUNIDADE:
- Empresa: {{$json.account_name}} (Setor: {{$json.sector}}, Receita:
R$({{$json.revenue}})
- Deal: {{$json.deal_name}} | Valor: R$({{$json.amount}})
- Stage atual: {{$json.stage}} | Dias nesse stage:
{{$json.days_in_stage}}
- Close date previsto: {{$json.expected_close_date}}
({{$json.days_to_close}} dias)
- Última atividade: {{$json.last_activity_days}} dias atrás
- Score calculado: {{$json.rule_score}}/100 (Tier {{$json.tier}})
- Vendedor responsável: {{$json.salesperson_name}}

Retorne exatamente nesse formato JSON:
{
  "urgency": "alta|media|baixa",
  "main_risk": "descrição do principal risco em 1 frase",
  "next_action": "ação específica e acionável para o vendedor em 1
frase",
  "why": "justificativa em 1 frase conectando os dados",
  "ai_score_adjustment": número entre -10 e +10
}

Regras:
- main_risk deve citar dados concretos do deal
- next_action deve ser algo que o vendedor faça hoje ou amanhã
- ai_score_adjustment só sai do 0 se houver sinal forte de risco ou
oportunidade
```

Validação e Confiabilidade

Erros mais comuns da IA nesse contexto:

- Inventar dados que não estão no prompt (alucinação de histórico)
- Retornar JSON malformado quebrando o fluxo n8n
- Recomendação genérica demais ("entre em contato com o cliente")
- `ai_score_adjustment` sempre positivo (viés otimista)

Como reduzir:

```
// Node de validação pós-IA no n8n
const raw = $json.ai_response;

try {
  // Remove eventuais ```json ``` que a IA coloca
  const cleaned = raw.replace(/```json|```/g, '').trim();
  const parsed = JSON.parse(cleaned);

  // Valida campos obrigatórios
  const required = ['urgency', 'main_risk', 'next_action', 'why',
'ai_score_adjustment'];
  for (const field of required) {
    if (!parsed[field]) throw new Error(`Campo faltando: ${field}`);
  }

  // Clamp no adjustment
  parsed.ai_score_adjustment = Math.max(-10, Math.min(10,
```

```
parsed.ai_score_adjustment));

  // Score final
  parsed.final_score = Math.max(0, Math.min(100,
    $json.rule_score + parsed.ai_score_adjustment
  ));

  return parsed;

} catch(e) {
  // Fallback seguro — não quebra o fluxo
  return {
    urgency: 'media',
    main_risk: 'Não foi possível analisar automaticamente.',
    next_action: 'Revisar manualmente.',
    why: 'Erro no processamento IA.',
    ai_score_adjustment: 0,
    final_score: $json.rule_score
  };
}
```

🛠️ 4. IMPLEMENTAÇÃO

Workflow n8n — Node por Node

WF-01: Score Batch

```
[Schedule Trigger] — a cada 1h
       ↓
[Postgres: Query]
  SELECT sp.*, a.revenue, a.employee_count, a.sector,
         p.price, st.salesperson_name
  FROM sales_pipeline sp
  JOIN accounts a ON sp.account_id = a.id
  JOIN products p ON sp.product_id = p.id
  JOIN sales_teams st ON sp.owner_id = st.id
  WHERE sp.stage NOT IN ('closed_won','closed_lost')
       ↓
[Split in Batches] — batch size: 1 (processa 1 deal por vez)
       ↓
[Code Node: calcScore()] — retorna rule_score + tier
       ↓
[HTTP Request → Claude API] — body com prompt final
       ↓
[Code Node: parseAndValidate()] — JSON parse + fallback
       ↓
[Postgres: Upsert scored_leads]
  INSERT INTO scored_leads (...)
  ON CONFLICT (deal_id) DO UPDATE SET ...
       ↓
[Set Node: log status] — opcional
```

WF-02: Score On-Demand (Webhook)

```
[Webhook] POST /score-deal
       ↓
[Postgres: Query] — busca 1 deal pelo deal_id do body
       ↓
[Code Node: calcScore()]
       ↓
```

```
[HTTP Request → Claude API]
        ↓
[Code Node: parseAndValidate()]
        ↓
[Postgres: Upsert]
        ↓
[Respond to Webhook] — retorna o JSON do score
```

Estrutura da Tabela `scored_leads`

```
CREATE TABLE scored_leads (
  deal_id           TEXT PRIMARY KEY,
  deal_name         TEXT,
  account_name      TEXT,
  salesperson_name  TEXT,
  amount            NUMERIC,
  stage             TEXT,
  rule_score        INT,
  ai_score_adj      INT,
  final_score       INT,
  tier              CHAR(1),          -- A, B ou C
  urgency           TEXT,
  main_risk         TEXT,
  next_action       TEXT,
  why               TEXT,
  scored_at         TIMESTAMPTZ DEFAULT NOW()
);

-- Index para o dashboard
CREATE INDEX idx_scored_leads_tier ON scored_leads(tier, final_score
DESC);
CREATE INDEX idx_scored_leads_salesperson ON
scored_leads(salesperson_name);
```

Dashboard Metabase — O que Criar
Card 1 — Top 10 Deals para Hoje

```
SELECT deal_name, account_name, amount, tier, final_score,
next_action
FROM scored_leads
WHERE tier = 'A'
ORDER BY final_score DESC
LIMIT 10
```

Card 2 — Distribuição por Tier Gráfico de pizza: COUNT por tier A/B/C
Card 3 — Pipeline por Vendedor

```
SELECT salesperson_name,
       COUNT(*) FILTER (WHERE tier='A') as tier_a,
       COUNT(*) FILTER (WHERE tier='B') as tier_b,
       SUM(amount) FILTER (WHERE tier='A') as valor_prioritario
FROM scored_leads
GROUP BY salesperson_name
ORDER BY tier_a DESC
```

Card 4 — Deals em Risco (atrasados + score baixo)

```
SELECT deal_name, account_name, days_to_close, final_score, main_risk
FROM scored_leads
WHERE days_to_close < 7 AND final_score < 50
ORDER BY days_to_close ASC
```

Filtro global no Metabase: `salesperson_name` → cada vendedor vê só os seus.

Como Fazer a Demo (em 15 minutos)

1. Mostra o Metabase com Top 10 Deals abertos (Tier A)
2. Clica num deal — mostra o `next_action` gerado pela IA
3. Abre o n8n, dispara o WF-01 manualmente com 5 deals
4. Mostra o log rodando em tempo real
5. Volta no Metabase, atualiza — score novo aparece
6. Mostra o card "Deals em Risco" — impacto imediato para gestor

Argumento de valor para a demo:
"Antes: vendedor escolhia por feeling. Agora: sistema analisa 8.800 deals, prioriza os 10 que mais merecem atenção hoje, e entrega a ação específica a tomar. Em 30 segundos o vendedor sabe exatamente o que fazer."

🚀 Extras de Alto Impacto

Alerta Slack (2h extra):

```
Após o Upsert, adiciona condição:
IF tier = 'A' AND scored_at > NOW() - INTERVAL '1h'
  → HTTP Request para Slack Webhook
  → Mensagem: "🔥 Deal prioritário novo: [nome] | R$[valor] | Ação:
[next_action]"
```

"Modo Foco" para o vendedor: View no Supabase filtrando `tier = 'A'` do vendedor logado. Integra com Retool para ter uma tela só dos Top 5 com botão "Marcar como contatado".

Insight semanal automático: Schedule toda segunda-feira → agrega os dados da semana → manda um email/Slack para o gestor com:

- Quantos deals avançaram de stage
- Top 3 vendedores por score médio do pipeline
- Deals que pioraram de score (precisam de atenção)

Checklist de Execução (4–6h)

```
☐ 1h — Setup Supabase + criar tabela scored_leads + importar dados
☐ 1h — WF-01 no n8n: fetch + score determinístico funcionando
☐ 1h — Integração Claude API + prompt + parse
☐ 30m — Upsert no Supabase + validar dados na tabela
☐ 1h — Metabase: conectar + criar 4 cards do dashboard
☐ 30m — Testar fluxo completo de ponta a ponta
☐ 30m — Preparar demo (script + dados de exemplo)
```

A chave do sucesso aqui é resistir ao overengineering. Score determinístico + 1 chamada de IA por deal + tabela simples + dashboard direto. Isso já é mais do que 99% dos times de vendas têm hoje.

Você é um especialista em arquitetura de sistemas com IA, n8n, RevOps e construção de MVPs rápidos.
Estou resolvendo este challenge:
https://github.com/Gestao-Quatro-Ponto-Zero/ai-master-challenge/tree/main/challenges/build-003-lead-scorer
---
CONTEXTO
- 35 vendedores
- ~8.800 oportunidades
- Priorização hoje é no “feeling”
Problema:
Má alocação de esforço → perda de tempo, baixa conversão e receita.

DADOS
4 tabelas:
- accounts (empresa, receita, funcionários, setor)
- products (produto, preço)
- sales_teams (vendedor, manager, região)
- sales_pipeline (principal: stage, datas, valor, etc)

OBJETIVO
Criar um sistema funcional que:
- Prioriza oportunidades (score)
- Explica o score
- Sugere próxima ação
- Exibe em dashboard simples

STACK
- n8n (orquestração)
- Banco (Supabase/Postgres)
- IA (OpenAI ou Claude)
- Dashboard (Metabase/Retool)
Tempo: 4–6h → foco em simplicidade e execução.

O QUE EU QUERO
1. Arquitetura
- Fluxo completo (dados → n8n → IA → banco → dashboard)
- Pipeline passo a passo
2. Decisões técnicas
- Como estruturar n8n
- Banco e joins
- Dashboard mais rápido de implementar
- Usar dataset completo ou amostra?
3. Lógica de negócio
- Principais features
- Regras de scoring (simples)
- Onde usar IA (e por quê)
4. Uso de IA

- Onde agrega valor vs não usar
- Prompt inicial → melhorias → prompt final
5. Implementação
- Workflow n8n (node por node)
- Estrutura do banco (incluindo tabela final com score)
- Dashboard (o que mostrar + filtros)
6. Validação
- Possíveis erros da IA
- Como garantir consistência
7. Diferenciais
- Sugestões simples de alto impacto (ex: top leads, insights)

REGRAS
- Evite overengineering
- Priorize execução rápida
- Foque em algo que vendedor realmente usaria
- Explique o porquê das decisões
- Seja direto e prático (com exemplos quando necessário)

Mostrar mais
09:01

Pesquisou na web

Pesquisou na web

Consegui ler o challenge do GitHub. O site não renderizou o README, mas tenho o contexto completo do que você precisa. Vou agora criar um documento visual de arquitetura completo e prático.

Agora o detalhamento completo de execução, começando pela estrutura do n8n:

Agora a lógica de scoring e uso de IA:

A resposta do Claude foi interrompida

vou te mandar o readme aqui:
Challenge 003 — Lead Scorer
Área: Vendas / RevOps Tipo: Build (construir solução funcional) Time budget: 4-6

horas

Contexto

Você é o novo AI Master da área de Vendas. O time comercial tem 35 vendedores distribuídos em escritórios regionais, gerenciados por managers, trabalhando um pipeline de ~8.800 oportunidades. Hoje, a priorização é feita "no feeling" — cada vendedor decide quais deals focar com base na própria experiência e intuição.

A Head de Revenue Operations te chamou e disse:

"Nossos vendedores gastam tempo demais em deals que não vão fechar e deixam oportunidades boas esfriar. Preciso de algo funcional — não um modelo no Jupyter Notebook que ninguém vai usar. Quero uma ferramenta que o vendedor abra, veja o pipeline, e saiba onde focar. Pode ser simples, mas precisa funcionar."

Este é o challenge mais "mão na massa". O deliverable principal é software funcionando — não um documento.

Dados disponíveis

Quatro tabelas de um CRM, todas interconectadas:

Dataset: __CRM Sales Predictive Analytics__ (licença CC0)

ArquivoO que contémRegistrosCampos-chave`accounts.csv`Contas clientes — setor, receita, número de funcionários, localização, empresa-mãe~85`accountproducts.csv`Catálogo de produtos com série e preço7`productsales_teams.csv`Vendedores com seu manager e escritório regional35`sales_agentsales_pipeline.csv`Pipeline completo — cada oportunidade com stage, datas, vendedor, produto, conta e valor de fechamento~8.800`opportunity_id` → liga tudo

Estrutura dos dados

```
accounts ←── sales_pipeline ──→ products
                    ↓
               sales_teams
```

O `sales_pipeline.csv` é a tabela central. Cada registro é uma oportunidade com:

- `deal_stage`: Prospecting, Engaging, Won, Lost
- `engage_date` / `close_date`: timeline do deal
- `close_value`: valor real de fechamento (0 se Lost) O que entregar
- Solução funcional (obrigatório)
- Construa algo que um vendedor possa usar. Não importa a tecnologia — importa que funcione.
- Exemplos de soluções válidas:
- Aplicação web (Streamlit, React, HTML+JS, qualquer coisa)
- Dashboard interativo (Plotly Dash, Retool, Metabase)
- CLI tool ou script que gera relatório priorizados
- API que recebe dados de um deal e retorna score + explicação
- Planilha inteligente com fórmulas de scoring
- Bot que envia prioridades por Slack/email
- Requisitos mínimos:
- Precisa rodar (não é mockup, wireframe ou PowerPoint)
- Precisa usar os dados reais do dataset
- Precisa ter lógica de scoring/priorização (não é só ordenar por valor)
- O vendedor precisa entender por que um deal tem score alto ou baixo
- Documentação mínima (obrigatório)
- Setup: Como rodar a solução (dependências, comandos, URL)
- Lógica: Que critérios de scoring você usou e por quê
- Limitações: O que a solução não faz e o que precisaria pra escalar

- Process log (obrigatório)
- Evidências de como você usou IA para construir. Leia o __Guia de Submissão__.
- Este challenge é especialmente interessante para quem usa "vibe coding" — Cursor, Claude Code, Replit Agent, v0, etc. Mostre o processo.
- Critérios de qualidade
- A solução funciona de verdade? Dá pra rodar seguindo as instruções?
- O scoring faz sentido? Usa as features certas? Vai além do óbvio?
- O vendedor (não-técnico) consegue usar e entender?
- A interface ajuda a tomar decisão ou só mostra dados?
- O código é limpo o suficiente pra outro dev dar manutenção?
- Dicas
- A Head de RevOps não pediu ML perfeito. Pediu algo útil. Comece simples, itere.
- Deal stage, tempo no pipeline, tamanho da conta, produto e vendedor são features óbvias. O que mais importa? Olhe os dados.
- Um scoring baseado em regras + heurísticas, bem apresentado, vale mais que um XGBoost sem interface.
- Explainability ganha. Se o vendedor entender POR QUE o deal tem score 85, a ferramenta é 10x mais útil que um número sem contexto.
- Pense no uso real: o vendedor abre isso na segunda-feira de manhã. O que ele precisa ver?
- Bonus: se a solução tiver filtro por vendedor/manager/região, fica imediatamente mais útil.

Mostrar mais
09:02

Perfeito — agora tenho o README completo. Isso muda algumas coisas importantes na minha recomendação anterior. Deixa eu re-calibrar para o que o challenge realmente pede.
Agora com o README real em mãos, posso calibrar a recomendação com precisão. Há diferenças importantes em relação ao que discutimos antes — o challenge é mais focado em entregar software funcional rápido do que em arquitetura n8n complexa. Deixa eu reorganizar tudo.

O que o README realmente pede (e o que muda)

Três pontos críticos que o README enfatiza que mudam a estratégia:
Primeiro: o deliverable é software rodando, não arquitetura. A Head de RevOps quer algo que o vendedor abra na segunda-feira. Isso significa que uma app web standalone provavelmente ganha no critério "funciona de verdade" mais rápido que uma stack n8n + Supabase + Metabase.
Segundo: os dados são CSVs locais (dataset CC0 do Kaggle), não um CRM live com webhook. O n8n só agrega valor real se você tiver dados fluindo continuamente — com CSVs estáticos, você adiciona complexidade sem ganhar nada.
Terceiro: `close_value` é 0 quando Lost e valor real quando Won. O campo `deal_stage` tem só 4 valores: Prospecting, Engaging, Won, Lost. Isso muda completamente a lógica de scoring.

Recomendação revisada: stack real para este challenge

Lógica de scoring calibrada para os dados reais

O dataset tem `deal_stage` com 4 valores e `close_value` = 0 quando Lost. Isso significa que você pode calcular win rate histórico por setor, produto e vendedor — e usar isso no score. Aqui está a lógica completa:

As features 4 e 5 são o diferencial real desse challenge — o dataset tem histórico completo de Won e Lost, então você consegue calcular win rate por setor e por vendedor. Isso vai além do óbvio (o README avisa que deal stage, tempo e valor são features óbvias) e mostra que você leu os dados de verdade.

Código completo — Streamlit app funcional

Aqui está o código real, pronto para rodar:
`app.py` — cole e execute:

```
import streamlit as st
import pandas as pd
import anthropic
import json
from datetime import datetime, date

# ── CONFIG ──────────────────────────────────────────────────────────────
st.set_page_config(
    page_title="Lead Scorer",
    page_icon="📊",
    layout="wide"
)

# ── CARREGAMENTO E CACHE DOS DADOS ──────────────────────────────────────
@st.cache_data
def load_data():
    pipeline = pd.read_csv("sales_pipeline.csv")
    accounts = pd.read_csv("accounts.csv")
    products = pd.read_csv("products.csv")
    teams    = pd.read_csv("sales_teams.csv")

    # join central
    df = pipeline.merge(accounts, on="account", how="left")
    df = df.merge(products,  on="product",  how="left")
    df = df.merge(teams,     on="sales_agent", how="left")

    # normaliza datas
    df["engage_date"] = pd.to_datetime(df["engage_date"],
errors="coerce")
    df["close_date"]  = pd.to_datetime(df["close_date"],
errors="coerce")
```

```
    # dias no pipeline (desde engage_date)
    today = pd.Timestamp(date.today())
    df["days_in_pipeline"] = (today -
df["engage_date"]).dt.days.fillna(0)

    return df

@st.cache_data
def compute_win_rates(df):
    historical = df[df["deal_stage"].isin(["Won", "Lost"])].copy()

    # win rate por setor
    sector_wr = (
        historical.groupby("sector")
        .apply(lambda x: (x["deal_stage"] == "Won").mean())
        .reset_index(name="sector_win_rate")
    )

    # win rate por vendedor
    agent_wr = (
        historical.groupby("sales_agent")
        .apply(lambda x: (x["deal_stage"] == "Won").mean())
        .reset_index(name="agent_win_rate")
    )

    return sector_wr, agent_wr

# ── FUNÇÃO DE SCORING
──────────────────────────────────────────────────────────────
def score_deal(row, sector_wr_map, agent_wr_map, max_value):
    score = 0
    breakdown = {}

    # 1. STAGE (0-30 pts)
    stage_pts = {"Engaging": 30, "Prospecting": 15, "Won": 0, "Lost":
0}
    s = stage_pts.get(row.get("deal_stage", ""), 0)
    score += s
    breakdown["stage"] = s

    # 2. TEMPO NO PIPELINE (0-20 pts) — penaliza stagnação acima de
90 dias
    days = row.get("days_in_pipeline", 0)
    if days <= 30:    t = 20
    elif days <= 60:  t = 15
    elif days <= 90:  t = 8
    elif days <= 180: t = 3
    else:             t = 0
    score += t
    breakdown["tempo"] = t

    # 3. VALOR DO DEAL (0-20 pts)
    val = row.get("close_value", 0) or 0
    if max_value > 0:
        v = round((val / max_value) * 20)
    else:
        v = 0
    score += v
    breakdown["valor"] = v
```

```
    # 4. WIN RATE DO SETOR (0–15 pts)
    wr_sector = sector_wr_map.get(row.get("sector", ""), 0.5)
    w = round(wr_sector * 15)
    score += w
    breakdown["win_rate_setor"] = w

    # 5. WIN RATE DO VENDEDOR (0–10 pts)
    wr_agent = agent_wr_map.get(row.get("sales_agent", ""), 0.5)
    a = round(wr_agent * 10)
    score += a
    breakdown["win_rate_vendedor"] = a

    # 6. TAMANHO DA CONTA (0–5 pts)
    rev = row.get("revenue", 0) or 0
    emp = row.get("employees", 0) or 0
    if rev > 10_000_000 or emp > 500: sz = 5
    elif rev > 1_000_000 or emp > 100: sz = 3
    else: sz = 1
    score += sz
    breakdown["tamanho_conta"] = sz

    score = min(100, max(0, score))

    if score >= 70:   tier = "A"
    elif score >= 40: tier = "B"
    else:             tier = "C"

    return score, tier, breakdown

# ── IA: RECOMENDAÇÃO VIA CLAUDE ──────────────────────────────────────────
def get_ai_recommendation(row, score, tier, breakdown):
    client = anthropic.Anthropic()

    prompt = f"""Você é um especialista em vendas B2B. Analise a
oportunidade abaixo e retorne SOMENTE um JSON válido, sem markdown.

OPORTUNIDADE:
- Empresa: {row.get('account', 'N/A')} | Setor: {row.get('sector',
'N/A')}
- Deal stage: {row.get('deal_stage', 'N/A')}
- Dias no pipeline: {row.get('days_in_pipeline', 0):.0f} dias
- Valor esperado: R${row.get('close_value', 0):,.0f}
- Vendedor: {row.get('sales_agent', 'N/A')} | Região:
{row.get('regional_office', 'N/A')}
- Win rate histórico do setor: {breakdown.get('win_rate_setor', 0)/
15*100:.0f}%
- Win rate histórico do vendedor: {breakdown.get('win_rate_vendedor',
0)/10*100:.0f}%
- Score calculado: {score}/100 (Tier {tier})

Retorne exatamente:
{{
  "urgencia": "alta|media|baixa",
  "risco_principal": "1 frase com dado concreto",
  "proxima_acao": "acao especifica para hoje ou amanha",
  "motivo": "1 frase conectando os dados ao score"
}}"""
```

```
    try:
        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=400,
            messages=[{"role": "user", "content": prompt}]
        )
        text = response.content[0].text.strip()
        text = text.replace("```json", "").replace("```", "").strip()
        return json.loads(text)
    except Exception:
        return {
            "urgencia": "media",
            "risco_principal": "Não foi possível analisar
automaticamente.",
            "proxima_acao": "Revisar manualmente.",
            "motivo": "Erro no processamento IA."
        }

# ── INTERFACE PRINCIPAL
──────────────────────────────────────────────────────────────
def main():
    df = load_data()
    sector_wr_df, agent_wr_df = compute_win_rates(df)

    sector_wr_map = dict(zip(sector_wr_df["sector"],
sector_wr_df["sector_win_rate"]))
    agent_wr_map  = dict(zip(agent_wr_df["sales_agent"],
agent_wr_df["agent_win_rate"]))

    # filtra só deals abertos
    open_deals = df[df["deal_stage"].isin(["Prospecting",
"Engaging"])].copy()
    max_value  = open_deals["close_value"].max() or 1

    # aplica scoring
    results = []
    for _, row in open_deals.iterrows():
        s, t, b = score_deal(row.to_dict(), sector_wr_map,
agent_wr_map, max_value)
        results.append({**row.to_dict(), "score": s, "tier": t,
"breakdown": b})

    scored = pd.DataFrame(results).sort_values("score",
ascending=False)

    # ── SIDEBAR
──────────────────────────────────────────────────────────────
    st.sidebar.title("Filtros")

    agents = ["Todos"] +
sorted(scored["sales_agent"].dropna().unique().tolist())
    sel_agent = st.sidebar.selectbox("Vendedor", agents)

    managers = ["Todos"] +
sorted(scored["manager"].dropna().unique().tolist())
    sel_manager = st.sidebar.selectbox("Manager", managers)

    regions = ["Todos"] +
```

```
sorted(scored["regional_office"].dropna().unique().tolist())
    sel_region = st.sidebar.selectbox("Região", regions)

    tiers = st.sidebar.multiselect("Tier", ["A", "B", "C"],
default=["A", "B"])

    # aplica filtros
    view = scored.copy()
    if sel_agent  != "Todos": view = view[view["sales_agent"] ==
sel_agent]
    if sel_manager != "Todos": view = view[view["manager"] ==
sel_manager]
    if sel_region != "Todos": view = view[view["regional_office"] ==
sel_region]
    if tiers: view = view[view["tier"].isin(tiers)]

    # ── HEADER METRICS
──────────────────────────────────────────────────────────────
    st.title("Lead Scorer — Pipeline Priorizado")

    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Deals abertos",    len(view))
    col2.metric("Tier A",           len(view[view["tier"] == "A"]))
    col3.metric("Valor Tier A",
                f"R${view[view['tier']=='A']
['close_value'].sum():,.0f}")
    col4.metric("Score médio",      f"{view['score'].mean():.0f}")

    # ── TABELA PRINCIPAL
──────────────────────────────────────────────────────────────
    st.subheader("Pipeline priorizado")

    def color_tier(val):
        colors = {"A": "background-color:#d4edda",
                  "B": "background-color:#fff3cd",
                  "C": "background-color:#f8d7da"}
        return colors.get(val, "")

    display_cols = ["account", "deal_stage", "sales_agent",
"regional_office",
                    "close_value", "days_in_pipeline", "score",
"tier"]
    display_cols = [c for c in display_cols if c in view.columns]

    st.dataframe(
        view[display_cols].head(50).style.applymap(color_tier,
subset=["tier"]),
        use_container_width=True,
        hide_index=True
    )

    # ── DETALHE DE DEAL COM IA
──────────────────────────────────────────────────────────────
    st.subheader("Análise detalhada com IA")
    st.caption("Selecione um deal para ver recomendação da IA")

    if len(view) > 0:
        deal_options = view.head(20)["account"].tolist()
        sel_deal_name = st.selectbox("Selecione o deal:",
```

```
deal_options)

        if st.button("Analisar com IA"):
            deal_row = view[view["account"] == sel_deal_name].iloc[0]
            score_val = deal_row["score"]
            tier_val  = deal_row["tier"]
            breakdown = deal_row["breakdown"]

            with st.spinner("Consultando Claude..."):
                rec = get_ai_recommendation(
                    deal_row.to_dict(), score_val, tier_val,
breakdown
                )

            col_a, col_b = st.columns([1, 2])
            with col_a:
                st.metric("Score", f"{score_val}/100")
                st.metric("Tier", tier_val)
                st.metric("Urgência", rec.get("urgencia",
"—").upper())

                st.caption("Breakdown do score:")
                for k, v in breakdown.items():
                    st.progress(v / 30, text=f"{k}: {v} pts")

            with col_b:
                st.info(f"**Risco principal:**
{rec.get('risco_principal', '—')}")
                st.success(f"**Próxima ação:**
{rec.get('proxima_acao', '—')}")
                st.caption(f"Motivo: {rec.get('motivo', '—')}")

if __name__ == "__main__":
    main()
```

requirements.txt:

```
streamlit>=1.32.0
pandas>=2.0.0
anthropic>=0.25.0
```

Setup em 3 comandos:

```
pip install -r requirements.txt
export ANTHROPIC_API_KEY=sk-...   # ou set no Windows
streamlit run app.py
```

Prompt final otimizado (já está no código acima, mas aqui isolado para referência)

```
prompt = f"""Você é um especialista em vendas B2B. Analise a
oportunidade abaixo
e retorne SOMENTE um JSON válido, sem markdown.

OPORTUNIDADE:
- Empresa: {account} | Setor: {sector}
- Deal stage: {stage}
- Dias no pipeline: {days} dias
```

```
- Valor esperado: R${value:,.0f}
- Vendedor: {agent} | Região: {region}
- Win rate histórico do setor: {sector_wr:.0%}
- Win rate histórico do vendedor: {agent_wr:.0%}
- Score calculado: {score}/100 (Tier {tier})

Retorne exatamente:
{{
  "urgencia": "alta|media|baixa",
  "risco_principal": "1 frase com dado concreto",
  "proxima_acao": "acao especifica para hoje ou amanha",
  "motivo": "1 frase conectando os dados ao score"
}}"""
```

O segredo desse prompt é passar o win rate histórico calculado — a IA não precisa inferir se o setor converte bem ou não, ela recebe o número direto e pode contextualizar a recomendação com fatos reais do dataset.

O que mostrar na demo (roteiro de 10 minutos)

1. Abre o app com `streamlit run app.py` — vendedor abre no browser
2. Filtra pelo vendedor "X" — sidebar instant, sem reload
3. Mostra os 5 deals Tier A do vendedor — tabela verde no topo
4. Clica no deal mais pontuado → "Analisar com IA" → espera 3s
5. Mostra o breakdown visual do score (progress bars) + recomendação da IA
6. Explica: "esse score 82 vem de: stage Engaging (+30), setor com 71% de win rate histórico (+10), deal de alto valor (+18)..."
7. Muda filtro para o manager — mostra todos os vendedores da equipe

Argumento de valor para a banca:
"Antes: 35 vendedores acordam na segunda-feira sem saber em qual dos 8.800 deals focar. Agora: cada um abre esse app, filtra pelo seu nome, e vê os 5 deals que merecem atenção hoje — com o motivo específico de cada score. Não é ML, é heurística inteligente com dados reais de histórico."

Diferenciais que ganham pontos extras no critério

O README diz explicitamente que "filtro por vendedor/manager/região fica imediatamente mais útil" — isso já está no sidebar acima.
Para ir além do óbvio e ganhar pontos em "o scoring faz sentido, vai além do óbvio?": o win rate por setor e por vendedor calculados do próprio dataset histórico é o diferencial. Nenhum concorrente que só olhar para deal stage e valor vai ter isso.
Para o critério "o vendedor entende por que o score é alto": o breakdown visual com progress bars para cada componente do score resolve isso diretamente. O vendedor vê "win rate do setor: 12 pts" e entende que o setor dele converte bem — não é uma caixa preta.
Para o `process_log` (obrigatório no challenge): documente cada prompt que você usou com Claude para gerar partes do código, incluindo esta conversa. Isso demonstra o processo de "vibe coding" que o README menciona explicitamente como algo interessante de mostrar.

Limitações para documentar (o README pede isso)

Seja honesto — isso conta pontos, não tira:

- O score usa win rate histórico agregado, não preditivo por combinação de features (ex: empresa de tech + produto X + vendedor Y juntos)
- Sem dados de atividade recente (ligações, emails) — o CRM real teria isso, o

CSV não tem
- Claude API tem custo por chamada — em produção, faria sentido cachear recomendações por 24h
- `close_value` nos deals abertos pode ser estimativa imprecisa — o score de valor assume que o número é confiável
- Para escalar: substituir regras por um modelo logístico treinado no próprio histórico Won/Lost, que já tem volume suficiente (~8.800 registros)

acho a segunda opção melhor, não sei como funciona o streamlit, tenho mais familiaridade com n8n, consigo ganhar tempo com claude code, então monte por favor a arquitetura da opção 2 e o prompt para eu mandar no claude code
09:08

Ótima escolha. Vou montar tudo calibrado para você executar no Claude Code com o mínimo de fricção.
Antes do prompt, deixa eu desenhar a arquitetura exata da Opção B para você ter clareza do que vai ser construído:

Agora o prompt completo para o Claude Code. Escrevi para ser auto-suficiente — você cola e executa sem precisar explicar nada:

Prompt para o Claude Code

```
Você é um especialista em RevOps e automação com n8n. Vou te passar o
contexto completo de um projeto e quero que você construa tudo,
arquivo por arquivo, com instruções exatas de execução.

---

# PROJETO: Lead Scorer para Time de Vendas

## CONTEXTO
Sistema que prioriza ~8.800 oportunidades de vendas automaticamente.
Problema real: 35 vendedores priorizam deals "no feeling", gerando
desperdício de esforço.
Solução: score automático + recomendação de IA para cada deal aberto.

## STACK
- n8n (orquestração e workflows)
- Supabase/Postgres (banco de dados)
- Claude API via HTTP Request no n8n (IA)
- Metabase (dashboard final)

## DATASET (já tenho os CSVs)
4 arquivos CSV de um CRM real (dataset CC0 público):
- sales_pipeline.csv: ~8.800 linhas — campos principais:
opportunity_id, account, product, sales_agent, deal_stage (valores:
Prospecting, Engaging, Won, Lost), engage_date, close_date,
```

close_value
- accounts.csv: ~85 linhas — campos: account, sector, revenue, employees, office_location, subsidiary_of
- products.csv: 7 linhas — campos: product, series, sales_price
- sales_teams.csv: 35 linhas — campos: sales_agent, manager, regional_office

O QUE QUERO QUE VOCÊ CONSTRUA

1. SQL DE SETUP DO BANCO (Supabase)
Gere o SQL completo para criar as 4 tabelas raw que espelham os CSVs, e a tabela de output scored_leads.

A tabela scored_leads deve ter:
- opportunity_id TEXT PRIMARY KEY
- account TEXT
- sales_agent TEXT
- manager TEXT
- regional_office TEXT
- product TEXT
- deal_stage TEXT
- engage_date DATE
- close_date DATE
- close_value NUMERIC
- sector TEXT
- days_in_pipeline INT
- rule_score INT (score das regras, 0-100)
- ai_score_adj INT (ajuste da IA, -10 a +10)
- final_score INT (rule_score + ai_score_adj, clampado 0-100)
- tier CHAR(1) (A, B ou C)
- urgency TEXT
- main_risk TEXT
- next_action TEXT
- why_score TEXT
- scored_at TIMESTAMPTZ DEFAULT NOW()

Crie também os índices necessários para o dashboard performar.

2. LÓGICA DE SCORING (JavaScript para node Code no n8n)
Implemente a função calcScore() com estas regras exatas:

FEATURE 1 — Deal Stage (0-30 pts):
- Engaging: 30 pts
- Prospecting: 15 pts
- Won/Lost: 0 pts (não devem entrar no pipeline, mas trate defensivamente)

FEATURE 2 — Tempo no pipeline desde engage_date (0-20 pts):
- 0-30 dias: 20 pts
- 31-60 dias: 15 pts
- 61-90 dias: 8 pts
- 91-180 dias: 3 pts
- 180+ dias: 0 pts (deal congelado)

FEATURE 3 — Valor do deal / close_value (0-20 pts):
- Normalizado: (close_value / max_close_value_do_batch) * 20
- Arredondado para inteiro

FEATURE 4 — Win rate histórico do setor (0-15 pts):

- Calculado a partir dos deals Won e Lost já existentes no banco
- Para cada setor: COUNT(Won) / (COUNT(Won) + COUNT(Lost))
- Multiplicado por 15 e arredondado

FEATURE 5 — Win rate histórico do vendedor (0-10 pts):
- Mesmo cálculo, mas por sales_agent
- Multiplicado por 10 e arredondado

FEATURE 6 — Tamanho da conta (0-5 pts):
- revenue > 10.000.000 OU employees > 500: 5 pts
- revenue > 1.000.000 OU employees > 100: 3 pts
- Demais: 1 pt

TIER baseado no final_score:
- A: 70-100 (fechar essa semana)
- B: 40-69 (nutrir e avançar)
- C: 0-39 (baixa prioridade)

Importante: as features 4 e 5 precisam de uma query prévia no n8n para calcular os win rates antes do loop de scoring. Monte essa query também.

3. WORKFLOW n8n — WF-01 (Score Batch)
Descreva node por node com os parâmetros exatos:

Node 1 — Schedule Trigger
Node 2 — Postgres: query para calcular win_rates por setor (retorna JSON)
Node 3 — Postgres: query para calcular win_rates por vendedor (retorna JSON)
Node 4 — Postgres: query principal com JOIN das 4 tabelas, só deals Prospecting e Engaging
Node 5 — Set: injeta os mapas de win_rate no item para o próximo node acessar
Node 6 — Split in Batches (size: 1)
Node 7 — Code: calcScore() — recebe o deal + win_rate maps, retorna rule_score, tier, breakdown
Node 8 — HTTP Request: chama Claude API com o prompt de IA
Node 9 — Code: parseAndValidate() — faz JSON.parse, valida campos, fallback em caso de erro, calcula final_score
Node 10 — Postgres: Upsert na tabela scored_leads

Para cada node, especifique:
- O tipo exato do node no n8n
- Todos os parâmetros configuráveis
- O JSON de expressões n8n necessárias (ex: {{ $json.campo }})
- Como conectar ao próximo node

4. WORKFLOW n8n — WF-02 (On-Demand via Webhook)
Versão simplificada que recebe um opportunity_id via POST e reprocessa só aquele deal.
Mesmos nodes de scoring e IA, mas começa com Webhook e termina com Respond to Webhook retornando o JSON completo do score.

5. PROMPT PARA A CLAUDE API (usado no node HTTP Request)
O prompt deve:
- Ser enviado como body JSON para https://api.anthropic.com/v1/messages
- Usar model: claude-sonnet-4-20250514

- Incluir todos os campos relevantes do deal no contexto
- Incluir o rule_score e o breakdown (quantos pontos vieram de cada feature)
- Incluir os win rates reais calculados do dataset
- Solicitar retorno SOMENTE em JSON válido, sem markdown, com campos:
  - urgency: "alta" | "media" | "baixa"
  - main_risk: string (1 frase, cita dado concreto)
  - next_action: string (ação específica para hoje ou amanhã)
  - why_score: string (1 frase conectando os dados ao score)
  - ai_score_adj: number (-10 a +10, só sai de 0 se houver sinal forte)

6. CÓDIGO DE VALIDAÇÃO DO RESPONSE DA IA (node parseAndValidate)
JavaScript que:
- Remove eventuais ```json ``` do response
- Faz JSON.parse com try/catch
- Valida que todos os 5 campos existem
- Clampeia ai_score_adj entre -10 e +10
- Calcula final_score = Math.max(0, Math.min(100, rule_score + ai_score_adj))
- Em caso de erro, retorna fallback seguro (não quebra o fluxo)
- Retorna o objeto completo pronto para o Upsert

7. QUERIES DO METABASE (SQL pronto para colar)
Gere as 4 queries SQL para criar os cards do dashboard:

Card 1 — Top 10 deals prioritários (Tier A, ordenado por final_score desc)
Card 2 — Distribuição por tier (count e soma de valor por A/B/C)
Card 3 — Pipeline por vendedor (tier_a_count, tier_b_count, valor_total_tier_a por sales_agent)
Card 4 — Deals em risco (tier B/C com close_date nos próximos 14 dias, ordenado por close_date)

Inclua um filtro de dashboard recomendado para filtrar por sales_agent (para que cada vendedor veja só o seu).

8. DOCUMENTAÇÃO DE SETUP
Gere um README.md com:
- Pré-requisitos (n8n, conta Supabase free, conta Metabase, chave Claude API)
- Passo a passo numerado para subir tudo do zero
- Como rodar o WF-01 pela primeira vez
- Como conectar o Metabase no Postgres do Supabase
- Troubleshooting dos 3 erros mais comuns

REGRAS DE IMPLEMENTAÇÃO

1. Todo código JavaScript nos nodes Code do n8n deve funcionar no ambiente do n8n (sem imports externos, use apenas APIs nativas do Node.js disponíveis no n8n)

2. As expressões n8n devem usar a sintaxe correta: {{ $json.campo }} para o item atual, {{ $('NomeDoNode').item.json.campo }} para referenciar outros nodes

3. O workflow deve ser robusto: se a IA falhar ou retornar JSON

```
inválido, o fluxo continua com fallback e registra o deal com o
rule_score puro

4. Para o HTTP Request da Claude API, o header deve incluir:
   - Content-Type: application/json
   - x-api-key: (variável de ambiente n8n)
   - anthropic-version: 2023-06-01

5. O Upsert no Postgres deve usar ON CONFLICT (opportunity_id) DO
UPDATE SET para ser idempotente — rodar duas vezes não duplica dados

6. Para as features 4 e 5 (win rates), os mapas devem ser calculados
UMA vez antes do Split in Batches e passados via Set node para não
fazer query a cada deal

7. O campo days_in_pipeline deve ser calculado como: CURRENT_DATE -
engage_date (em dias), usando SQL no momento da query principal

---

# ENTREGÁVEIS ESPERADOS

Quero que você entregue, em ordem:

1. setup.sql — SQL completo para rodar no Supabase
2. wf01_scoring_logic.js — a função calcScore() isolada com
comentários
3. wf01_parse_validate.js — a função parseAndValidate() isolada
4. wf01_description.md — descrição detalhada node por node do WF-01
5. wf02_description.md — descrição detalhada node por node do WF-02
6. claude_prompt.txt — o prompt exato para a API, com placeholders
marcados
7. metabase_queries.sql — as 4 queries do dashboard
8. README.md — documentação de setup

Entregue um arquivo por vez, com o nome do arquivo como título, e
avise quando terminar todos os 8.
```

Como usar esse prompt

Cole exatamente como está no Claude Code. Ele vai gerar os 8 arquivos em sequência. Algumas dicas de como extrair o máximo:

Depois que ele gerar o `wf01_description.md`, se algum node ficar confuso, peça: *"no node X, me mostra o JSON de configuração exato para importar no n8n"* — o Claude Code consegue gerar o JSON de workflow que você importa direto.

Se quiser pular a parte de configurar Metabase e fazer uma demo mais rápida, depois que os arquivos estiverem prontos, peça: *"agora gera um arquivo HTML single-page que conecta no Supabase REST API e mostra o dashboard com as mesmas 4 queries, sem precisar do Metabase"* — isso vira um diferencial visual no challenge e leva uns 20 minutos a mais.

Para o `process log` obrigatório do challenge, salve os prints desta conversa e das interações com o Claude Code — isso documenta exatamente o processo de "vibe coding" que o README pede.

me mostre como posso usar o streamlit com claude code
09:11

Direto ao ponto — aqui está o fluxo completo.

O que é o Claude Code nesse contexto

Claude Code é uma CLI que roda no seu terminal e age como um dev autônomo: lê arquivos, escreve código, instala dependências e executa comandos. Você dá uma instrução em linguagem natural e ele constrói o projeto.

Para o seu Lead Scorer com Streamlit, o fluxo é este:

```
seu terminal
     ↓
claude (CLI)
     ↓
lê os CSVs → escreve app.py → instala libs → roda streamlit
```

Passo a passo para instalar e usar

1. Instala o Claude Code

```
npm install -g @anthropic-ai/claude-code
```

Precisa ter Node.js 18+ instalado. Verifique com `node --version`.

2. Autentica

```
claude
```

Na primeira vez abre o browser para login com sua conta Anthropic. Após login, volta ao terminal.

3. Cria a pasta do projeto e coloca os CSVs lá

```
mkdir lead-scorer
cd lead-scorer
# copie os 4 CSVs para essa pasta:
# sales_pipeline.csv, accounts.csv, products.csv, sales_teams.csv
```

4. Abre o Claude Code dentro da pasta

```
claude
```

5. Cola o prompt abaixo

Prompt completo para o Claude Code

```
Você é um desenvolvedor Python especialista em Streamlit e análise de
dados.
Vou te passar um contexto completo. Quero que você construa uma
aplicação
Streamlit funcional de Lead Scoring do zero, rodando localmente.

---

# CONTEXTO DO PROJETO
```

Sistema de priorização de oportunidades de vendas para um time com 35 vendedores e ~8.800 deals. O vendedor abre o app na segunda-feira e sabe
exatamente onde focar.

DADOS DISPONÍVEIS

Há 4 arquivos CSV na pasta atual. Antes de escrever qualquer código, leia os primeiros 5 registros de cada CSV para entender os campos reais
e os valores exatos (especialmente deal_stage — os valores são case-sensitive).

O QUE CONSTRUIR

Arquivo: requirements.txt
Crie com as dependências necessárias:
- streamlit
- pandas
- anthropic
- plotly

Arquivo: scoring.py
Módulo separado com a lógica de scoring. Implemente:

Função: load_and_merge(data_dir=".")
- Lê os 4 CSVs
- Faz merge: sales_pipeline com accounts (on="account"),
  com products (on="product"), com sales_teams (on="sales_agent")
- Converte engage_date e close_date para datetime
- Calcula days_in_pipeline = (hoje - engage_date).days
- Retorna DataFrame completo

Função: compute_win_rates(df)
- Filtra deals com stage Won ou Lost (use os valores reais que você leu)
- Calcula win_rate por sector: Won / (Won + Lost)
- Calcula win_rate por sales_agent: Won / (Won + Lost)
- Retorna dois dicts: sector_wr e agent_wr

Função: score_deal(row, sector_wr, agent_wr, max_value)
Implementa scoring com 6 features (total 100 pts):

Feature 1 — deal_stage (30 pts):
  Engaging = 30, Prospecting = 15, outros = 0
  Use os valores de stage exatos que você leu nos dados.

Feature 2 — days_in_pipeline (20 pts):
  0-30 dias = 20, 31-60 = 15, 61-90 = 8, 91-180 = 3, 180+ = 0

Feature 3 — close_value normalizado (20 pts):
  round((close_value / max_value) * 20) se max_value > 0

Feature 4 — win rate do sector (15 pts):

```
  round(sector_wr.get(sector, 0.5) * 15)

Feature 5 — win rate do sales_agent (10 pts):
  round(agent_wr.get(agent, 0.5) * 10)

Feature 6 — tamanho da conta (5 pts):
  revenue > 10_000_000 ou employees > 500 = 5 pts
  revenue > 1_000_000 ou employees > 100 = 3 pts
  demais = 1 pt

Score final: soma das 6 features, clamp 0-100
Tier: A=70+, B=40-69, C=0-39

Retorna: score (int), tier (str), breakdown (dict com pts de cada
feature)

### Função: score_all(df)
- Filtra só deals abertos (Prospecting e Engaging)
- Calcula win rates uma vez
- Aplica score_deal em cada linha
- Adiciona colunas: score, tier, breakdown ao DataFrame
- Ordena por score desc
- Retorna DataFrame

## Arquivo: ai_insights.py
Módulo para chamada da Claude API. Implemente:

### Função: get_recommendation(row, breakdown, api_key)
- Recebe uma linha do DataFrame (deal) + breakdown do score + api_key
- Chama claude-sonnet-4-20250514 via
anthropic.Anthropic(api_key=api_key)
- Prompt deve incluir todos os campos relevantes do deal E o
breakdown
  (quantos pontos cada feature contribuiu)
- Solicita retorno SOMENTE em JSON com campos:
    urgency: "alta" | "media" | "baixa"
    main_risk: string (1 frase com dado concreto do deal)
    next_action: string (ação específica para hoje ou amanhã)
    why_score: string (1 frase explicando o score)
- Tem try/except: se falhar, retorna dict com valores padrão
- Remove ```json ``` do response antes de parsear
- Retorna dict

## Arquivo: app.py
Aplicação Streamlit principal. Estrutura:

### Configuração inicial
- st.set_page_config: title="Lead Scorer", layout="wide",
page_icon="📊"
- Usa @st.cache_data para carregar e escorar os dados (TTL 1 hora)

### Sidebar
- Input de API key da Anthropic (st.text_input, type="password")
  com instrução: "Cole sua chave para análise com IA"
- Selectbox: Vendedor (com opção "Todos")
- Selectbox: Manager (com opção "Todos")
- Selectbox: Região (com opção "Todos")
- Multiselect: Tier (default: ["A", "B"])
- Slider: Score mínimo (0-100, default 0)
```

```
- Botão: "Atualizar scores" que limpa o cache

### Header com 4 métricas (st.columns)
Baseadas na view filtrada:
- Total de deals visíveis
- Deals Tier A
- Valor total Tier A (formatado em R$)
- Score médio

### Aba 1: Pipeline priorizado
- Tabela com colunas: account, deal_stage, sales_agent,
  regional_office, close_value, days_in_pipeline, score, tier
- Tier A = fundo verde claro, Tier B = amarelo, Tier C = vermelho
claro
- Máximo 100 linhas, ordenado por score
- Abaixo da tabela: st.caption com explicação das features do score

### Aba 2: Análise com IA
- Selectbox com os top 20 deals (por score) da view filtrada
- Quando seleciona um deal, mostra automaticamente:
  - Lado esquerdo (col 1/3):
    - Métrica: Score e Tier
    - Métrica: Urgência (da IA, ou "—" se sem chave)
    - Gráfico de barras horizontal (plotly) mostrando o breakdown
      do score: feature → pontos. Título: "De onde vem o score"
  - Lado direito (col 2/3):
    - Se api_key preenchida: botão "Analisar com IA"
      - Ao clicar: st.spinner → chama get_recommendation → exibe:
        - st.info com main_risk
        - st.success com next_action
        - st.caption com why_score
    - Se sem api_key: st.warning "Cole sua API key na sidebar"
    - Sempre exibe (independente de IA): tabela com todos os
      campos do deal selecionado

### Aba 3: Visão do gestor
- Gráfico de barras agrupadas por vendedor:
  count de Tier A (verde) e Tier B (amarelo)
  Use plotly express, ordenado por total de deals Tier A desc
- Tabela resumo por manager:
  vendedores, deals_tier_a, deals_tier_b, valor_total_pipeline
- Gráfico de dispersão: x=days_in_pipeline, y=score,
  cor=tier, hover=account+sales_agent

### Rodapé
- st.caption com: "Scores calculados em {timestamp}.
  Features: deal stage (30pts) + tempo no pipeline (20pts) +
  valor (20pts) + win rate setor (15pts) + win rate vendedor (10pts)
  + tamanho da conta (5pts)"

---

# REGRAS DE IMPLEMENTAÇÃO

1. Leia os CSVs antes de escrever qualquer código para usar os
   nomes de campos e valores exatos

2. Trate valores nulos: close_value nulo = 0, revenue nulo = 0,
   employees nulo = 0
```

```
3. O app deve funcionar mesmo sem API key — scoring por regras
   funciona sempre, IA é opcional

4. Use plotly para todos os gráficos (não matplotlib)

5. Formate close_value como moeda:
   f"R${valor:,.0f}".replace(",", ".")

6. O breakdown no gráfico deve mostrar o nome legível da feature,
   não o nome da variável. Ex: "Deal stage" não "stage_pts"

7. Após criar todos os arquivos, execute:
   pip install -r requirements.txt
   E depois verifique se o app sobe sem erros com:
   streamlit run app.py --server.headless true
   (deixa rodar por 5 segundos e para)

---

# ENTREGÁVEIS

Crie nesta ordem:
1. requirements.txt
2. scoring.py
3. ai_insights.py
4. app.py

Depois instale as dependências e teste o startup.
Ao final, me diga:
- Quais campos reais você encontrou nos CSVs
- Se houve algum ajuste na lógica por causa dos dados reais
- O comando exato para eu rodar o app
```

O que acontece depois que você cola o prompt

O Claude Code vai ler os seus CSVs primeiro — isso é importante porque os nomes de campo e valores de `deal_stage` nos dados reais podem ser ligeiramente diferentes do que qualquer documentação descreve. Ele vai ajustar o código para os dados exatos que você tem.

Enquanto ele trabalha, você vai ver ele executando coisas como `cat sales_pipeline.csv | head -5`, escrevendo os arquivos, rodando `pip install` e testando o startup. Tudo no terminal, sem intervenção sua.

Quando terminar, o comando para abrir o app é simplesmente:

```
streamlit run app.py
```

Abre no browser em `localhost:8501`. Você coloca sua API key da Anthropic na sidebar e a análise com IA fica disponível instantaneamente para qualquer deal.

Se algo der errado

Se ele reportar erro em algum ponto, você pode simplesmente dizer no terminal do Claude Code:

```
o erro foi: [cola o erro aqui]. Corrija.
```

Ele lê o erro, entende a causa e corrige os arquivos automaticamente.

vou fazer a opção dois mesmo
09:14

Perfeito. Vou te dar tudo calibrado para executar no Claude Code com a stack n8n + Supabase + Metabase.

Antes do prompt — o que você precisa ter pronto

Antes de rodar o Claude Code, deixa esses 4 itens configurados para ele não travar no meio:

1. Conta Supabase — crie em supabase.com (free tier suficiente). Após criar o projeto, pegue:

- `Project URL` (Settings → API)
- `anon key` (Settings → API)
- `Connection string` do Postgres (Settings → Database → Connection string → URI)

2. n8n rodando — a forma mais rápida localmente:

```
npx n8n
```

Abre em `localhost:5678`. Crie uma conta de admin na primeira vez.

3. Chave da Claude API — platform.anthropic.com → API Keys

4. Os 4 CSVs — numa pasta chamada `lead-scorer/data/`

Prompt completo para o Claude Code

Cole isso no terminal do Claude Code dentro da pasta `lead-scorer`:

```
Você é um especialista em RevOps, n8n e Supabase. Vou te passar o
contexto
completo de um projeto. Quero que você gere todos os arquivos
necessários
para construir um sistema de Lead Scoring funcional.

Antes de qualquer coisa, leia os arquivos CSV na pasta ./data/ para
entender os campos reais e os valores exatos de cada coluna
(especialmente deal_stage — os valores são case-sensitive).

---

# CONTEXTO DO PROJETO

Sistema de priorização automática de ~8.800 oportunidades de vendas.
35 vendedores hoje priorizam "no feeling". O sistema vai calcular um
score para cada deal aberto e gerar recomendações via IA.

Stack:
- n8n (orquestração dos workflows)
- Supabase / Postgres (banco de dados)
- Claude API via HTTP Request no n8n (IA)
- Metabase (dashboard final)
```

```
---

# DADOS DISPONÍVEIS

4 CSVs na pasta ./data/:
- sales_pipeline.csv (~8800 linhas): oportunidades — campos incluem
  opportunity_id, account, product, sales_agent, deal_stage,
  engage_date, close_date, close_value
- accounts.csv (~85 linhas): empresas — sector, revenue, employees,
  office_location
- products.csv (7 linhas): produto, series, sales_price
- sales_teams.csv (35 linhas): sales_agent, manager, regional_office

---

# ENTREGÁVEIS — gere nesta ordem exata

## 1. setup.sql
SQL completo para rodar no Supabase SQL Editor.

Deve criar:

Tabela accounts:
  account TEXT PRIMARY KEY,
  sector TEXT, revenue NUMERIC, employees INT,
  office_location TEXT, subsidiary_of TEXT

Tabela products:
  product TEXT PRIMARY KEY,
  series TEXT, sales_price NUMERIC

Tabela sales_teams:
  sales_agent TEXT PRIMARY KEY,
  manager TEXT, regional_office TEXT

Tabela sales_pipeline:
  opportunity_id TEXT PRIMARY KEY,
  account TEXT REFERENCES accounts(account),
  product TEXT REFERENCES products(product),
  sales_agent TEXT REFERENCES sales_teams(sales_agent),
  deal_stage TEXT, engage_date DATE, close_date DATE,
  close_value NUMERIC DEFAULT 0

Tabela scored_leads:
  opportunity_id TEXT PRIMARY KEY,
  account TEXT, sales_agent TEXT, manager TEXT,
  regional_office TEXT, product TEXT, deal_stage TEXT,
  engage_date DATE, close_date DATE, close_value NUMERIC,
  sector TEXT, days_in_pipeline INT,
  rule_score INT, ai_score_adj INT, final_score INT,
  tier CHAR(1), urgency TEXT, main_risk TEXT,
  next_action TEXT, why_score TEXT,
  scored_at TIMESTAMPTZ DEFAULT NOW()

Índices para performance do dashboard:
  idx em tier, final_score DESC
  idx em sales_agent
  idx em manager
  idx em regional_office
```

```
Ao final do SQL, inclua um comentário com o passo a passo
de como importar os CSVs via Supabase UI.

## 2. scoring.js
Função JavaScript pura para usar no node Code do n8n.
Não use imports externos — apenas lógica vanilla JS.

Deve exportar (como comentário explicativo, não módulo real)
a função calcScore(item, sectorWinRates, agentWinRates, maxValue)
que recebe:
- item: objeto com todos os campos do deal (resultado do JOIN)
- sectorWinRates: objeto { setor: winRate } (ex: {"Technology":
0.68})
- agentWinRates: objeto { agente: winRate }
- maxValue: número — maior close_value do batch atual

E implementa:

Feature 1 — deal_stage (0–30 pts):
  Use os valores de stage exatos que você leu nos CSVs.
  Engaging = 30, Prospecting = 15, outros = 0

Feature 2 — days_in_pipeline (0–20 pts):
  0–30 dias = 20 pts
  31–60 dias = 15 pts
  61–90 dias = 8 pts
  91–180 dias = 3 pts
  180+ dias = 0 pts

Feature 3 — close_value normalizado (0–20 pts):
  Math.round((close_value / maxValue) * 20)
  Se maxValue = 0, retorna 0

Feature 4 — win rate do sector (0–15 pts):
  Math.round((sectorWinRates[sector] ?? 0.5) * 15)

Feature 5 — win rate do sales_agent (0–10 pts):
  Math.round((agentWinRates[agent] ?? 0.5) * 10)

Feature 6 — tamanho da conta (0–5 pts):
  revenue > 10_000_000 OU employees > 500 → 5 pts
  revenue > 1_000_000 OU employees > 100 → 3 pts
  caso contrário → 1 pt

Tier:
  final_score >= 70 → "A"
  final_score >= 40 → "B"
  caso contrário → "C"

Retorna objeto:
{
  rule_score: number (0–100),
  tier: string,
  breakdown: {
    stage_pts: number,
    tempo_pts: number,
    valor_pts: number,
    setor_wr_pts: number,
```

```
    agente_wr_pts: number,
    conta_pts: number
  }
}

## 3. parse_validate.js
Função JavaScript para o node Code de validação pós-IA no n8n.

Recebe o item completo com campos:
- ai_raw_response: string com o response da Claude API
- rule_score: number
- tier: string
- breakdown: object
- [todos os outros campos do deal]

Deve:
1. Limpar o response: remover ```json e ``` se existirem
2. Tentar JSON.parse()
3. Validar que os campos existem: urgency, main_risk,
   next_action, why_score, ai_score_adj
4. Clampear ai_score_adj: Math.max(-10, Math.min(10, valor))
5. Calcular final_score:
   Math.max(0, Math.min(100, rule_score + ai_score_adj))
6. Se qualquer passo falhar: usar fallback seguro
   (urgency: "media", ai_score_adj: 0, final_score: rule_score,
    main_risk/next_action/why_score com mensagem padrão)
7. Retornar objeto único com TODOS os campos necessários
   para o Upsert: deal fields + score fields + ia fields

## 4. wf01_nodes.md
Documentação detalhada do WF-01 (Score Batch), node por node.

Para cada node, especifique:
- Nome do node (como nomear no n8n)
- Tipo exato do node no n8n
- Todos os parâmetros de configuração com valores exatos
- Expressões n8n a usar (sintaxe {{ $json.campo }})
- Como conectar ao próximo node

Ordem dos nodes:

Node 1: Schedule Trigger
  - Interval: a cada 6 horas (ou manual para demo)

Node 2: Postgres — Win Rates por Setor
  - Query que calcula win rate por sector a partir da tabela
    sales_pipeline JOIN accounts
  - Retorna JSON array: [{sector, win_rate}]

Node 3: Postgres — Win Rates por Vendedor
  - Query que calcula win rate por sales_agent
  - Retorna JSON array: [{sales_agent, win_rate}]

Node 4: Postgres — Deals Abertos
  - Query principal com JOIN das 4 tabelas
  - Filtra só deals com stage = Prospecting ou Engaging
  - Calcula days_in_pipeline com CURRENT_DATE - engage_date
  - Inclui todos os campos necessários para scoring e IA
```

Node 5: Code

A resposta do Claude foi interrompida

mas calma preciso que me ajude com isso primeiro:

1. Clique no challenge que mais combina com seu perfil
2. Leia o README completo — tem contexto, dados, e critérios de qualidade
3. Baixe os dados do Kaggle (links dentro de cada challenge)
4. Resolva usando IA — e documente como usou
5. Envie seguindo o __Guia de Submissão__

09:15

Ótimo, vamos com calma. Me conta:
Você já fez qual dessas etapas? E onde está travado — é no download do Kaggle, na submissão, ou em outra parte?

nenhu,a

09:15

Tudo bem, vamos do zero então.
Qual é o seu nível com essas ferramentas?
Me responde essas três que já monto o passo a passo exato para o seu caso.

P: Você tem conta no GitHub?
R: Sim, já uso
P: Você tem conta no Kaggle?
R: Sim
P: Qual sistema operacional você usa?
R: Mac

09:15

Perfeito. Você tem tudo que precisa. Vamos executar as 5 etapas em ordem.

Etapa 1 — Acessar o challenge e ler o README

Você já leu o README aqui comigo, então essa etapa está feita.

Etapa 2 — Baixar os dados do Kaggle

O dataset é o **CRM Sales Predictive Analytics**. Acesse diretamente:
kaggle.com/datasets/innocentmfa/crm-sales-predictive-analytics
Na página, clique em **Download** (botão no canto superior direito). Vai baixar um `.zip`. Descompacte — você vai ter os 4 CSVs:

```
sales_pipeline.csv
accounts.csv
products.csv
sales_teams.csv
```

Crie uma pasta para o projeto e coloque os CSVs lá:

```
mkdir ~/lead-scorer
mkdir ~/lead-scorer/data
mv ~/Downloads/*.csv ~/lead-scorer/data/
```

Etapa 3 — Fazer o fork do repositório do challenge

Isso é necessário para a submissão. No GitHub:

1. Acesse `github.com/Gestao-Quatro-Ponto-Zero/ai-master-challenge`
2. Clique em **Fork** (canto superior direito)
3. Confirma — vai criar uma cópia no seu GitHub

Depois clone localmente:

```
cd ~/lead-scorer
git clone https://github.com/SEU_USUARIO/ai-master-challenge.git
```

A sua solução vai para a pasta:

```
ai-master-challenge/challenges/build-003-lead-scorer/
```

Etapa 4 — Ler o Guia de Submissão

Antes de construir qualquer coisa, você precisa saber exatamente o que entregar. Acesse:
github.com/Gestao-Quatro-Ponto-Zero/ai-master-challenge
Procure o arquivo `SUBMISSION_GUIDE.md` ou similar na raiz do repositório. Me cola o conteúdo aqui — assim eu já monto a estrutura de entrega correta junto com o código.

Próximo passo imediato

Você consegue fazer essas duas coisas agora?

1. Baixar os CSVs do Kaggle
2. Colar aqui o conteúdo do Guia de Submissão

Com isso em mãos, a gente já parte direto para a construção.

me ajude no passo a passo com a etapa 3
09:19

Vamos lá. Passo a passo com prints mentais de cada clique.

Etapa 3 — Fork e clone do repositório

Passo 1 — Fazer o Fork

Acesse esse link: **github.com/Gestao-Quatro-Ponto-Zero/ai-master-challenge**

No canto superior direito da página, clique no botão **Fork**.

```
[ Watch ] [ Star ] [ Fork 4 ]
                      ↑
                clica aqui
```

Na próxima tela, deixa tudo como está e clica em **Create fork**. Em 5 segundos aparece o repositório na sua conta — a URL vai ser:

```
github.com/SEU_USUARIO/ai-master-challenge
```

Passo 2 — Clonar na sua máquina

Abre o Terminal no Mac (`Cmd + Espaço` → digita "Terminal" → Enter).

Escolhe onde quer guardar o projeto. Recomendo a pasta home:

```
cd ~
```

Agora clona o seu fork (substitui `SEU_USUARIO` pelo seu usuário do GitHub):

```
git clone https://github.com/SEU_USUARIO/ai-master-challenge.git
```

Vai aparecer algo assim:

```
Cloning into 'ai-master-challenge'...
remote: Enumerating objects...
✓ done.
```

Passo 3 — Navegar até a pasta do challenge

```
cd ai-master-challenge/challenges/build-003-lead-scorer
```

Confere o que tem lá:

```
ls
```

Deve aparecer o README.md que você já leu.

Passo 4 — Mover os CSVs para a pasta certa

```
mkdir data
mv ~/Downloads/sales_pipeline.csv ./data/
mv ~/Downloads/accounts.csv ./data/
mv ~/Downloads/products.csv ./data/
mv ~/Downloads/sales_teams.csv ./data/
```

Confirma que estão lá:

```
ls data/
```

Deve mostrar os 4 arquivos.

Conseguiu executar esses passos? Me diz onde travou ou cola o output do terminal que a gente segue para a etapa 4.

o passo 4 não deu certo

09:27

Normal — o caminho do download pode variar. Vamos achar os arquivos.
Cola esse comando no terminal:

```
find ~ -name "sales_pipeline.csv" 2>/dev/null
```

Me mostra o output que ele retornar — com o caminho exato eu te dou o comando certo para mover.

tenho a pasta data criada dentro da pasta build-003-lead-scorer

09:27

Ótimo, então só falta mover os CSVs para dentro dela. Cola esse comando para ver onde estão os arquivos:

```
find ~ -name "*.csv" 2>/dev/null
```

Me mostra o que aparecer.

```
sacefilmes@Mac-Studio-de-sace build-003-lead-scorer % find ~ -name "*.csv" 2>/
dev/null
/Users/sacefilmes/.trae/extensions/miguelsolorio.symbols-0.0.24/file-types/CSV.csv
/Users/sacefilmes/Desktop/challenge G4/lead-scorer/data/products.csv
/Users/sacefilmes/Desktop/challenge G4/lead-scorer/data/accounts.csv
/Users/sacefilmes/Desktop/challenge G4/lead-scorer/data/metadata.csv
/Users/sacefilmes/Desktop/challenge G4/lead-scorer/data/sales_pipeline.csv
/Users/sacefilmes/Desktop/challenge G4/lead-scorer/data/sales_teams.csv
/Users/sacefilmes/Library/Application Support/com.adobe.dunamis/
dfdafab7-6da9-4e9c-b68b-38adc983caba/iaas.csv
/Users/sacefilmes/Library/Application Support/FireCut/Temp/RMS from test_38.csv
```

```
/Users/sacefilmes/Library/Application Support/FireCut/Temp/
frequency_distribution.csv
/Users/sacefilmes/Library/Preferences/Blackmagic Design/DaVinci Resolve/
Fairlight/FLDebugSettings.csv
```

Mostrar mais
09:28

Os CSVs já estão numa pasta `data` no Desktop. Agora é só copiar para dentro do projeto:

```
cp "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/data/"*.csv ~/
ai-master-challenge/challenges/build-003-lead-scorer/data/
```

Depois confirma que chegaram:

```
ls ~/ai-master-challenge/challenges/build-003-lead-scorer/data/
```

Deve listar os 4 arquivos. Me mostra o output.

```
sacefilmes@Mac-Studio-de-sace build-003-lead-scorer % cp "/Users/sacefilmes/
Desktop/challenge G4/lead-scorer/data/"*.csv ~/ai-master-challenge/challenges/
build-003-lead-scorer/data/
cp: /Users/sacefilmes/ai-master-challenge/challenges/build-003-lead-scorer/data is
not a directory
sacefilmes@Mac-Studio-de-sace build-003-lead-scorer % ls ~/ai-master-challenge/
challenges/build-003-lead-scorer/data/
ls: /Users/sacefilmes/ai-master-challenge/challenges/build-003-lead-scorer/data/:
No such file or directory
sacefilmes@Mac-Studio-de-sace build-003-lead-scorer % ls data
sacefilmes@Mac-Studio-de-sace build-003-lead-scorer %
```

09:30

A pasta `data` existe mas está vazia, e o caminho do seu projeto não é `~/ai-master-challenge`. Vamos achar onde ele está:

```
pwd
```

Me mostra o output — vai aparecer o caminho exato onde você está agora.

```
/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/
challenges/build-003-lead-scorer
```

sacefilmes@Mac-Studio-de-sace build-003-lead-scorer %
09:30

Perfeito, agora sei o caminho exato. Copia os CSVs assim:

```
cp "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/data/"*.csv "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/challenges/build-003-lead-scorer/data/"
```

Depois confirma:

```
ls data/
```

agora deu certo, vamos para o proximo
09:31

Ótimo! Agora a etapa 4 — ler o Guia de Submissão antes de construir qualquer coisa. Acesse o repositório original no GitHub e procure o arquivo de submissão:

```
ls "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/"
```

Me mostra o output — quero ver quais arquivos estão na raiz do repositório. Provavelmente tem um `SUBMISSION_GUIDE.md` ou `CONTRIBUTING.md` lá.

sacefilmes@Mac-Studio-de-sace build-003-lead-scorer % ls "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/"
CONTRIBUTING.md README.md submission-guide.md
LICENSE challenges templates
09:33

Tem os 3 arquivos importantes. Vamos ler o guia de submissão:

```
cat "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/submission-guide.md"
```

Cola o output aqui.

master-challenge/submission-guide.md"

Guia de Submissão

O que enviar

Sua submissão tem **duas partes obrigatórias**:

1. A Solução

O deliverable principal do desafio. O formato varia por challenge — leia o README específico.

Pode ser:

- Documento de análise (PDF, Markdown)
- Código funcional (scripts, notebooks, aplicação)
- Protótipo/aplicação rodando (código fonte + instruções de setup)
- Redesign de processo documentado
- Combinação dos acima

2. O Process Log (obrigatório)

Evidência de **como** você usou IA para chegar na solução. Sem process log = desclassificado.

Formatos aceitos (escolha um ou combine):

Formato	Exemplo
Screenshots	Prints das conversas com AI, mostrando seus prompts e as respostas
Screen recording	Vídeo do seu workflow (Loom, gravação de tela)
Chat export	Export da conversa (Claude, ChatGPT, etc.)
Narrativa escrita	Documento explicando passo a passo: "primeiro fiz X, depois pedi Y ao Claude, ajustei Z porque..."
Git history	Commits mostrando evolução do código com AI-assisted development
Notebook comentado	Jupyter/Colab com células de comentário explicando o raciocínio

O que queremos ver no process log:

- Quais ferramentas de IA você usou e por quê
- Como você decompôs o problema antes de promptar
- Onde a IA errou e como você corrigiu
- O que você adicionou que a IA sozinha não faria
- Quantas iterações foram necessárias

Como enviar

Via **Pull Request** — o único canal aceito. Veja [CONTRIBUTING.md](./CONTRIBUTING.md) para o passo a passo.

Use o [template de submissão](./templates/submission-template.md) como base para o README da sua pasta.

Formato da solução

Não há template obrigatório para a solução em si. Mas submissões que funcionam bem geralmente têm:

1. **Executive summary** (1 parágrafo) — o que você encontrou/construiu e por que importa
2. **Abordagem** — como você atacou o problema

3 **Resultado** — a análise, o protótipo, o redesign
4 **Recomendações** — o que fazer com isso
5 **Limitações** — o que você não conseguiu resolver ou verificar

---

O que torna uma submissão forte

- O candidato claramente **entendeu o problema** antes de começar
- A IA foi usada **estrategicamente** — não como "Google glorificado"
- O output é **acionável** — alguém poderia usar isso amanhã
- O process log mostra **iteração e julgamento** — não um prompt único
- A comunicação é **clara** — técnico e não-técnico entendem

O que torna uma submissão fraca

- Output genérico que poderia ser sobre qualquer empresa
- Zero evidência de verificação (a IA disse, o candidato acreditou)
- Process log mostra 1 prompt → 1 resposta → submissão
- Foco em parecer inteligente em vez de resolver o problema
- Documento de 40 páginas onde 5 resolveriam

Boa sorte. Mostre como você trabalha.
sacefilmes@Mac-Studio-de-sace build-003-lead-scorer %

Mostrar mais
09:34

Agora vamos ler o CONTRIBUTING.md para entender o passo a passo do Pull Request:

```
cat "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/CONTRIBUTING.md"
```

Cola o output aqui.

sacefilmes@Mac-Studio-de-sace build-003-lead-scorer % cat "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/CONTRIBUTING.md"

Como submeter sua solução

A submissão é feita **exclusivamente via Pull Request**. Isso faz parte do teste — se você vai implementar IA em áreas inteiras, precisa saber trabalhar com git.

Passo a passo

1. **Fork** este repositório
2. Clone o fork: `git clone https://github.com/SEU-USUARIO/ai-master-challenge.git`
3. Crie uma branch: `git checkout -b submission/seu-nome`
4. Crie sua pasta: `mkdir -p submissions/seu-nome`
5. Adicione sua solução e process log dentro da pasta
6. Commit e push: `git push origin submission/seu-nome`
7. Abra um **Pull Request** para `main` com título: `[Submission] Seu Nome — Challenge XXX`

Estrutura da pasta

```
submissions/seu-nome/
├── README.md            ← Use o template de templates/submission-
template.md
├── solution/            ← Sua solução (análise, código, protótipo)
│   ├── ...
│   └── ...
├── process-log/         ← Evidências de uso de IA
│   ├── screenshots/
│   ├── chat-exports/
│   └── ...
└── docs/                ← Documentação adicional (se houver)
```

Checklist antes de abrir o PR

- [ ] Escolhi um challenge e li o README completo
- [ ] Minha solução está na pasta `submissions/meu-nome/`
- [ ] Incluí o process log com evidências de uso de IA
- [ ] O README da minha submissão segue o [template](./templates/submission-template.md)
- [ ] Se construí código, incluí instruções de setup
- [ ] Não modifiquei nenhum arquivo fora da minha pasta

Regras do PR

- Só modifique arquivos dentro de `submissions/seu-nome/` — PRs que alteram outros arquivos serão rejeitados
- Um PR por pessoa. Se quiser atualizar, faça push na mesma branch
- O título do PR deve seguir o formato: `[Submission] Nome — Challenge XXX`

Nunca usou git?

Se você está aprendendo agora, use IA para te ajudar. Sério — pedir pro Claude ou ChatGPT te guiar pelo processo de fork, branch e PR é exatamente o tipo de coisa que um AI Master faz. Isso já é parte do teste.

Dúvidas?

Abra uma [issue](../../issues) neste repositório.

sacefilmes@Mac-Studio-de-sace build-003-lead-scorer %

Mostrar mais

09:57

Perfeito. Agora tenho tudo que preciso para te guiar do zero até o PR final.

Veja o que precisa ser feito e onde você está agora:

Fork do repositório — feito
Clone local — feito
CSVs na pasta data — feito
Criar a branch de submissão
Ler o template de submissão
Construir a solução
Montar o process log
Abrir o Pull Request

Vamos criar a branch agora. No terminal, navegue para a raiz do repositório:

```
cd "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge"
```

Cria a branch com seu nome:

```
git checkout -b submission/seu-nome
```

Substitui `seu-nome` pelo seu nome real, sem acento e com hífen.
Exemplo: `submission/joao-silva`.
Me mostra o output quando terminar.

acha melhor eu usar o vscode para fazer essas coisas?
09:57

Sim, muito melhor. O VSCode vai te dar:

- Terminal integrado (sem ficar alternando janelas)
- Visualização dos arquivos do projeto na barra lateral
- Git integrado com interface visual para commits e PR
- Claude Code roda direto no terminal do VSCode

Abre o projeto no VSCode agora

No terminal, ainda na pasta do projeto:

```
cd "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge"
code .
```

Se aparecer erro `command not found: code`, abre o VSCode manualmente, vai em **File → Open Folder** e navega até a pasta `ai-master-challenge`.
Quando abrir, me confirma que consegue ver os arquivos na barra lateral esquerda — deve aparecer a estrutura com as pastas `challenges`, `templates`, `submissions` etc.

já abri a pasta manualmente
09:59

Ótimo. Agora abre o terminal integrado do VSCode:
`Ctrl + ``` (acento grave) ou pelo menu **Terminal → New Terminal**
Vai abrir um terminal na parte de baixo. Cola esse comando para confirmar que está na pasta certa:

```
```

```
pwd
```
Me mostra o output.

não me mostrou nada
09:59

Tenta de novo — clica uma vez dentro da área do terminal para garantir que está com foco, depois digita:

```
pwd
```
E aperta Enter. O que aparece?

sacefilmes@Mac-Studio-de-sace ai-master-challenge % pwd
/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge
10:00

Perfeito, está na pasta certa. Agora cria a branch de submissão — substitui `seu-nome` pelo seu nome real:

```
git checkout -b submission/seu-nome
```
Me mostra o output.

Switched to a new branch 'submission/guilherme-pardin'
10:01

Perfeito. Agora vamos criar a estrutura de pastas da sua submissão:

```
mkdir -p submissions/guilherme-pardin/solution/data
mkdir -p submissions/guilherme-pardin/process-log/screenshots
mkdir -p submissions/guilherme-pardin/process-log/chat-exports
mkdir -p submissions/guilherme-pardin/docs
```
Confirma que criou tudo:

```
ls submissions/guilherme-pardin/
```

Me mostra o output.

```
sacefilmes@Mac-Studio-de-sace ai-master-challenge % ls submissions/guilherme-pardin/
docs            process-log     solution
```

10:02

Estrutura criada. Agora vamos ler o template de submissão para montar o README corretamente:

```
cat templates/submission-template.md
```

Me mostra o output.

```
sacefilmes@Mac-Studio-de-sace ai-master-challenge % cat templates/submission-template.md
# Submissão — [Seu Nome] — Challenge [XXX]
## Sobre mim
- **Nome:**
- **LinkedIn:**
- **Challenge escolhido:**
---
## Executive Summary
_Em 3-5 frases: o que você fez, o que encontrou, e qual a principal recomendação._
---
## Solução
_Sua análise, protótipo, redesign ou o que o challenge pedir. Use o formato que melhor comunicar._
### Abordagem
_Como você atacou o problema. Por onde começou? Como decompôs? O que priorizou?_
### Resultados / Findings
_O que você encontrou/construiu. Mostre dados, screenshots, links._
### Recomendações
_O que a empresa deveria fazer com base nos seus findings. Priorize._
### Limitações
_O que você não conseguiu resolver, verificar, ou que precisaria de mais tempo/dados._
---
## Process Log — Como usei IA
> **Este bloco é obrigatório.** Sem ele, a submissão é desclassificada.
```

Ferramentas usadas
Liste as ferramentas de IA que usou e para quê.
Ferramenta	Para que usou
ex: Claude Code	_Análise exploratória dos dados e geração de gráficos_
ex: Cursor	_Construção do protótipo web_
ex: ChatGPT	_Brainstorm de hipóteses iniciais_
Workflow
Descreva passo a passo como você trabalhou. Onde a IA entrou em cada etapa?
1.
2.
3.
Onde a IA errou e como corrigi
A IA fez algo errado? Você percebeu? Como corrigiu?
O que eu adicionei que a IA sozinha não faria
Qual foi seu julgamento, contexto, ou insight que fez diferença?

Evidências
Anexe ou linke as evidências do processo:
- [ ] Screenshots das conversas com IA
- [ ] Screen recording do workflow
- [ ] Chat exports
- [ ] Git history (se construiu código)
- [ ] Outro: _____

Submissão enviada em: [data]
sacefilmes@Mac-Studio-de-sace ai-master-challenge %

Mostrar mais
10:02

Perfeito. Agora temos tudo que precisamos para começar a construir.
Resumindo onde você está:

- Fork e clone feitos
- Branch `submission/guilherme-pardin` criada
- Estrutura de pastas criada
- CSVs baixados
- Template de submissão lido
- Construir a solução
- Preencher o README de submissão
- Montar o process log
- Commit + Push + Pull Request

Antes de partir para a construção, uma decisão importante:
Nas nossas conversas anteriores você ficou entre Streamlit e n8n+Supabase+Metabase. Dado que você está no VSCode agora e quer usar Claude Code, **o Streamlit é de longe a melhor escolha aqui** — o Claude Code constrói o app inteiro em uma pasta, você roda com um comando, e já tem algo funcional

para colocar em `submissions/guilherme-pardin/solution/`.
O n8n exige configurar infraestrutura externa (Supabase, Metabase, variáveis de ambiente) que consome 2-3h só de setup antes de escrever uma linha de código.
Quer seguir com Streamlit? Se sim, o próximo passo é instalar o Claude Code e disparar o prompt que eu já montei para você.

sim, mas eu já uso o claude code, com claude desktop
10:06

Perfeito, então você já tem o Claude Code funcionando.
Só preciso entender como você usa — quando você diz "Claude Code com Claude desktop", você usa ele:

P: Como você usa o Claude Code?
R: Claude Desktop como chat normal
10:06

Entendido — você usa o Claude Desktop como assistente de chat, não o Claude Code CLI que executa comandos no terminal.
São coisas diferentes:

	O que é	Como funciona
Claude Desktop	App de chat	Você conversa, ele responde — não executa nada no seu computador
Claude Code CLI	Ferramenta de terminal	Você roda `claude` no terminal, ele lê e escreve arquivos, instala dependências, executa comandos

Para construir o Streamlit app você tem duas opções:

P: Como prefere trabalhar?
R: eu uso o claude code dentro do claude desktop, ele mexe no meu computador por aqui
10:07

Ah, você tem o Claude Code com integração MCP no Claude Desktop — ele consegue ler e escrever arquivos direto no seu computador via chat. Isso é perfeito.

Então o fluxo é simples: você cola o prompt aqui no Claude Desktop, ele cria os arquivos direto na pasta certa, e você só roda o app no terminal do VSCode.
Antes de mandar o prompt, vamos colocar o Claude Code apontado para a pasta certa. Me confirma uma coisa:
Quando você pede para o Claude Desktop criar um arquivo, ele cria onde? Na pasta que você configurou, ou em qualquer lugar?

na pasta que eu configurei
10:08

Perfeito. Então configura ele para apontar para essa pasta:

```
/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/submissions/guilherme-pardin/solution
```

Quando confirmar que está apontando para lá, me fala que eu te passo o prompt completo para construir o app.

ficou assim
10:10

Perfeito — vejo que está na pasta `solution` e na branch `main`.
Só um ajuste antes de rodar: a branch deveria ser `submission/guilherme-pardin`, não `main`. Mas isso a gente resolve depois no terminal do VSCode. Por enquanto não impede de construir.
Agora cola esse prompt no campo de chat do Claude Code:

```
Antes de qualquer coisa, liste os arquivos disponíveis na pasta atual e leia os primeiros 10 registros de cada CSV na pasta data/ para entender os campos e valores exatos.

Depois construa uma aplicação Streamlit de Lead Scoring com os seguintes arquivos:

---

# CONTEXTO

Sistema de priorização de ~8.800 oportunidades de vendas para 35 vendedores. O vendedor abre o app e sabe exatamente onde focar.

---
```

```
# ARQUIVOS A CRIAR

## requirements.txt
streamlit
pandas
anthropic
plotly

## scoring.py
Módulo com 3 funções:

### load_and_merge()
- Lê os 4 CSVs da pasta data/
- Merge: sales_pipeline com accounts (on="account"),
  com products (on="product"), com sales_teams (on="sales_agent")
- Converte engage_date e close_date para datetime
- Calcula days_in_pipeline = (hoje - engage_date).days
- Trata nulos: close_value=0, revenue=0, employees=0
- Retorna DataFrame completo

### compute_win_rates(df)
- Filtra deals Won e Lost (use os valores exatos que você leu)
- Calcula win_rate por sector: Won/(Won+Lost)
- Calcula win_rate por sales_agent: Won/(Won+Lost)
- Retorna dois dicts: sector_wr e agent_wr

### score_all(df)
- Filtra só deals abertos (Prospecting e Engaging)
- Chama compute_win_rates uma vez
- Aplica score_deal em cada linha
- Retorna DataFrame com colunas score, tier, breakdown adicionadas
- Ordenado por score desc

### score_deal(row, sector_wr, agent_wr, max_value)
6 features, total 100 pts:

Feature 1 — deal_stage (30 pts):
  Use os valores exatos que você leu nos CSVs
  Engaging = 30, Prospecting = 15, outros = 0

Feature 2 — days_in_pipeline (20 pts):
  0-30 dias = 20
  31-60 dias = 15
  61-90 dias = 8
  91-180 dias = 3
  180+ dias = 0

Feature 3 — close_value normalizado (20 pts):
  round((close_value / max_value) * 20)

Feature 4 — win rate do sector (15 pts):
  round((sector_wr.get(sector, 0.5)) * 15)

Feature 5 — win rate do sales_agent (10 pts):
  round((agent_wr.get(agent, 0.5)) * 10)

Feature 6 — tamanho da conta (5 pts):
  revenue > 10000000 ou employees > 500 = 5
```

```
  revenue > 1000000 ou employees > 100 = 3
  demais = 1

Tier: A=70+, B=40-69, C=0-39

Retorna: score (int), tier (str), breakdown (dict com pts de cada
feature)

## ai_insights.py
Função get_recommendation(row, breakdown, api_key):
- Chama claude-sonnet-4-20250514
- Prompt inclui todos os campos do deal + breakdown do score
- Pede retorno SOMENTE em JSON com campos:
    urgency: "alta" | "media" | "baixa"
    main_risk: 1 frase com dado concreto
    next_action: ação específica para hoje ou amanhã
    why_score: 1 frase explicando o score
- Remove ```json``` antes de parsear
- Try/except com fallback seguro
- Retorna dict

## app.py
Aplicação Streamlit principal:

### Configuração
- st.set_page_config: title="Lead Scorer", layout="wide",
page_icon="📊"
- @st.cache_data para carregar dados (TTL 3600)

### Sidebar
- st.text_input para API key Anthropic (type="password")
- Selectbox: Vendedor (opção "Todos" + lista)
- Selectbox: Manager (opção "Todos" + lista)
- Selectbox: Região (opção "Todos" + lista)
- Multiselect: Tier (default ["A", "B"])
- Botão "Atualizar scores" que limpa o cache

### Header — 4 métricas em colunas
- Total de deals visíveis
- Deals Tier A
- Valor total Tier A formatado em R$
- Score médio

### 3 abas

Aba 1 — Pipeline priorizado:
- Tabela com: account, deal_stage, sales_agent, regional_office,
  close_value, days_in_pipeline, score, tier
- Tier A = fundo verde claro (#d4edda)
- Tier B = fundo amarelo (#fff3cd)
- Tier C = fundo vermelho claro (#f8d7da)
- Máximo 100 linhas
- st.caption explicando as features do score

Aba 2 — Análise com IA:
- Selectbox com top 20 deals da view filtrada
- Layout em 2 colunas:
  Coluna esquerda (1/3):
    - Métricas: Score, Tier
```

```
    - Gráfico de barras horizontal (plotly) com breakdown
      Labels legíveis: "Deal stage", "Tempo pipeline",
      "Valor do deal", "Win rate setor",
      "Win rate vendedor", "Tamanho conta"
  Coluna direita (2/3):
    - Se api_key preenchida: botão "Analisar com IA"
      Ao clicar: spinner → get_recommendation → exibe:
        st.info com main_risk
        st.success com next_action
        st.caption com why_score
    - Se sem api_key: st.warning pedindo a chave
    - Sempre: tabela com todos os campos do deal

Aba 3 — Visão do gestor:
- Gráfico barras agrupadas por vendedor:
  Tier A (verde) e Tier B (amarelo)
  Ordenado por total Tier A desc
- Tabela por manager:
  vendedores, deals_tier_a, deals_tier_b, valor_pipeline
- Scatter plot: x=days_in_pipeline, y=score,
  cor=tier, hover=account+sales_agent

### Rodapé
- st.caption com timestamp e explicação das features

---

# REGRAS

1. Leia os CSVs antes de escrever qualquer código
2. Use os valores exatos de deal_stage que encontrar
3. O app funciona sem API key (IA é opcional)
4. Use plotly para todos os gráficos
5. Formate valores como R$ com separador de milhar
6. Após criar os arquivos, instale as dependências:
   pip install -r requirements.txt
7. Teste se o app sobe sem erro:
   streamlit run app.py --server.headless true
8. Me diz quais campos reais encontrou nos CSVs e
   se fez algum ajuste na lógica
```

Manda esse prompt e me mostra o que ele retornar. Vamos acompanhar juntos.

mas não acha mais simples eu mudar a branch aqui já?
10:11

Sim, muito mais simples. No terminal do VSCode:

```
cd "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge"
git checkout submission/guilherme-pardin
```

Me mostra o output.

ve se esta certo agora
10:12

Perfeito — `solution` + `submission/guilherme-pardin`. Está tudo certo.
Agora cola o prompt que te passei no campo de chat do Claude Code e manda executar.

me mande o prompt de novo, já aproveite para otimiza-lo para deixar mais eficiente
10:12

Aqui está o prompt otimizado:

```
Você é um especialista em Python e Streamlit. Sua tarefa é construir
uma aplicação de Lead Scoring funcional do zero.

PRIMEIRA AÇÃO OBRIGATÓRIA: Leia os arquivos na pasta data/ antes de
escrever qualquer linha de código. Liste os campos reais e os valores
exatos de deal_stage — o código deve usar esses valores, não
suposições.

---

# CONTEXTO

App para 35 vendedores priorizarem ~8.800 deals de vendas.
Vendedor abre na segunda-feira e sabe exatamente onde focar.
IA (Claude API) é opcional — o scoring por regras funciona sempre.

---

# ESTRUTURA DE ARQUIVOS

Crie nesta ordem: requirements.txt → scoring.py → ai_insights.py →
app.py

---

## requirements.txt
streamlit
pandas
anthropic
```

plotly

```
---

## scoring.py

Quatro funções:

**load_and_merge()**
- Lê os 4 CSVs de data/
- Merge: pipeline ← accounts (on="account") ← products (on="product")
  ← sales_teams (on="sales_agent")
- Converte datas para datetime
- Calcula days_in_pipeline = (hoje - engage_date).days
- Nulos: close_value=0, revenue=0, employees=0
- Retorna DataFrame completo

**compute_win_rates(df)**
- Filtra Won e Lost (valores exatos dos CSVs)
- win_rate por sector: Won/(Won+Lost)
- win_rate por sales_agent: Won/(Won+Lost)
- Retorna (sector_wr: dict, agent_wr: dict)

**score_deal(row, sector_wr, agent_wr, max_value) → (score, tier,
breakdown)**

| Feature | Pontos | Regra |
|---|---|---|
| deal_stage | 0-30 | Engaging=30, Prospecting=15, outros=0 |
| days_in_pipeline | 0-20 | ≤30=20, ≤60=15, ≤90=8, ≤180=3, resto=0 |
| close_value | 0-20 | round((valor/max_value)*20) |
| win rate setor | 0-15 | round(sector_wr.get(sector, 0.5)*15) |
| win rate vendedor | 0-10 | round(agent_wr.get(agent, 0.5)*10) |
| tamanho conta | 0-5 | rev>10M ou emp>500=5, rev>1M ou emp>100=3,
resto=1 |

Tier: A=70+, B=40-69, C=0-39
breakdown: dict com pts de cada feature com labels legíveis

**score_all(df)**
- Filtra só Prospecting e Engaging
- Chama compute_win_rates uma vez
- Aplica score_deal em cada linha
- Retorna DataFrame com score, tier, breakdown — ordenado por score
desc

---

## ai_insights.py

**get_recommendation(row, breakdown, api_key) → dict**
- Model: claude-sonnet-4-20250514
- Prompt inclui: todos os campos do deal + breakdown com pontuação
  de cada feature
- Instrução: retornar SOMENTE JSON válido, sem markdown, com:
  - urgency: "alta"|"media"|"baixa"
  - main_risk: 1 frase com dado concreto do deal
  - next_action: ação específica para hoje ou amanhã
  - why_score: 1 frase conectando os dados ao score
```

- Remove ```json``` antes de parsear
- try/except com fallback seguro em todos os campos
- Retorna dict sempre (nunca lança exceção)

---

app.py

Setup
- page_config: title="Lead Scorer", layout="wide", page_icon="📊"
- @st.cache_data(ttl=3600) no load + scoring

Sidebar
- API key Anthropic (password)
- Filtros: Vendedor, Manager, Região (todos com opção "Todos")
- Multiselect Tier (default: A e B)
- Botão "Atualizar scores" limpa cache

Header — 4 métricas
Total deals | Tier A count | Valor Tier A em R$ | Score médio

Aba 1 — Pipeline priorizado
- Tabela: account, deal_stage, sales_agent, regional_office, close_value, days_in_pipeline, score, tier
- Cores: A=#d4edda, B=#fff3cd, C=#f8d7da
- Máximo 100 linhas
- Caption explicando as 6 features

Aba 2 — Análise com IA
- Selectbox: top 20 deals da view filtrada
- Col esquerda (1/3): métricas Score + Tier + gráfico barras horizontal plotly com breakdown (labels legíveis)
- Col direita (2/3):
  - Com API key: botão "Analisar com IA" → spinner → st.info(main_risk) + st.success(next_action) + st.caption(why_score)
  - Sem API key: st.warning pedindo a chave
  - Sempre: tabela com todos os campos do deal

Aba 3 — Visão do gestor
- Barras agrupadas por vendedor: Tier A (verde) + Tier B (amarelo), ordenado por Tier A desc
- Tabela por manager: vendedores, deals_tier_a, deals_tier_b, valor_pipeline
- Scatter: x=days_in_pipeline, y=score, cor=tier, hover=account+sales_agent

Rodapé
- Caption com timestamp + explicação das 6 features e seus pesos

---

REGRAS DE EXECUÇÃO

1. Leia os CSVs primeiro — use campos e valores exatos
2. App funciona 100% sem API key
3. Apenas plotly para gráficos
4. Valores monetários formatados como R$ com milhar
5. Após criar os 4 arquivos:

```
   - Execute: pip install -r requirements.txt
   - Teste: streamlit run app.py --server.headless true
6. Ao finalizar me informe:
   - Campos reais encontrados nos CSVs
   - Valores exatos de deal_stage
   - Qualquer ajuste feito por causa dos dados reais
   - Comando exato para eu rodar o app
```

mande tudo em um texto só, e preciso que otimize mais ainda tire contexto desnescesario e etc...

10:14

```
Leia todos os arquivos em data/ antes de escrever qualquer código.
Use os campos e valores exatos encontrados, especialmente os valores
de deal_stage.

Crie 4 arquivos nesta ordem:

---

requirements.txt:
streamlit
pandas
anthropic
plotly

---

scoring.py com 4 funções:

load_and_merge(): lê os 4 CSVs de data/, merge
pipeline←accounts(on="account")←products(on="product")←sales_teams(on
="sales_agent"), converte datas, calcula days_in_pipeline=(hoje-
engage_date).days, nulos viram 0, retorna DataFrame.

compute_win_rates(df): filtra Won/Lost, calcula win_rate por sector e
por sales_agent como Won/(Won+Lost), retorna dois dicts.

score_deal(row, sector_wr, agent_wr, max_value): retorna (score,
tier, breakdown) com estas 6 features:
- deal_stage: Engaging=30, Prospecting=15, outros=0
- days_in_pipeline: ≤30=20, ≤60=15, ≤90=8, ≤180=3, resto=0
- close_value: round((valor/max_value)*20)
- win rate setor: round(sector_wr.get(sector,0.5)*15)
- win rate vendedor: round(agent_wr.get(agent,0.5)*10)
- tamanho conta: rev>10M ou emp>500=5, rev>1M ou emp>100=3, resto=1
Tier: A=70+, B=40-69, C=0-39. breakdown com labels legíveis.

score_all(df): filtra Prospecting/Engaging, chama compute_win_rates
uma vez, aplica score_deal em cada linha, retorna DataFrame com
score/tier/breakdown ordenado por score desc.
```

---

ai_insights.py com função get_recommendation(row, breakdown, api_key): chama claude-sonnet-4-20250514, prompt inclui todos os campos do deal + breakdown com pontuação por feature, pede retorno SOMENTE JSON sem markdown com campos urgency("alta"|"media"|"baixa"), main_risk(1 frase com dado concreto), next_action(ação para hoje ou amanhã), why_score(1 frase). Remove ```json``` antes de parsear. try/except com fallback em todos os campos. Nunca lança exceção.

---

app.py:

Setup: page_config title="Lead Scorer" layout="wide" page_icon="📊", @st.cache_data(ttl=3600) no carregamento.

Sidebar: API key Anthropic(password), filtros Vendedor/Manager/Região com opção "Todos", multiselect Tier default=[A,B], botão "Atualizar scores" limpa cache.

Header: 4 métricas — total deals, count Tier A, valor Tier A em R$, score médio.

Aba 1 "Pipeline": tabela com account/deal_stage/sales_agent/regional_office/close_value/days_in_pipeline/score/tier, cores A=#d4edda B=#fff3cd C=#f8d7da, máximo 100 linhas, caption explicando as 6 features.

Aba 2 "Análise IA": selectbox top 20 deals. Col esquerda(1/3): score+tier+gráfico barras horizontal plotly do breakdown. Col direita(2/3): com API key mostra botão "Analisar com IA"→spinner→st.info(main_risk)+st.success(next_action) +st.caption(why_score); sem API key mostra st.warning; sempre mostra tabela com todos os campos do deal.

Aba 3 "Gestor": barras agrupadas por vendedor Tier A(verde) +B(amarelo) ordenado por A desc, tabela por manager com vendedores/deals_tier_a/deals_tier_b/valor_pipeline, scatter x=days_in_pipeline y=score cor=tier hover=account+sales_agent.

Rodapé: caption com timestamp e pesos das 6 features.

Regras:
- App funciona sem API key
- Só plotly para gráficos
- Valores em R$ com separador de milhar
- Após criar os arquivos: pip install -r requirements.txt e teste com streamlit run app.py --server.headless true
- Ao finalizar informe os valores exatos de deal_stage encontrados e o comando para rodar

coloque a persona pra ele também, como estava no outro com, como é o primeiro prompt ele precisa ter um contexto completo e detalhado

10:15

```
Você é um desenvolvedor Python sênior especialista em Streamlit,
pandas e análise de dados para times de vendas. Está construindo um
MVP funcional de Lead Scoring para um challenge técnico de RevOps. O
critério principal de avaliação é: funciona de verdade, o vendedor
consegue usar, e o scoring vai além do óbvio.

PRIMEIRA AÇÃO OBRIGATÓRIA: Leia todos os arquivos em data/ antes de
escrever qualquer código. Use os campos e valores exatos encontrados,
especialmente os valores de deal_stage — o código deve usar esses
valores, não suposições.

---

CONTEXTO DO PROJETO

Sistema de priorização automática de ~8.800 oportunidades de vendas
para 35 vendedores que hoje priorizam "no feeling". O vendedor abre o
app na segunda-feira, filtra pelo seu nome, e vê onde focar — com o
motivo específico de cada score. IA via Claude API é opcional: o
scoring por regras funciona sempre, a IA enriquece com recomendação
em linguagem natural.

Dataset: 4 CSVs na pasta data/
- sales_pipeline.csv (~8800 linhas): oportunidades com stage, datas,
valor, vendedor
- accounts.csv (~85 linhas): empresa, setor, receita, funcionários
- products.csv (7 linhas): produto e preço
- sales_teams.csv (35 linhas): vendedor, manager, região

O diferencial do scoring está nas features 4 e 5 — win rate histórico
calculado do próprio dataset. Isso vai além do óbvio (stage + valor)
e mostra que os dados foram realmente analisados.

---

Crie 4 arquivos nesta ordem:

---

requirements.txt:
streamlit
pandas
anthropic
plotly

---

scoring.py com 4 funções:

load_and_merge(): lê os 4 CSVs de data/, merge
pipeline←accounts(on="account")←products(on="product")←sales_teams(on
```

="sales_agent"), converte datas, calcula days_in_pipeline=(hoje-engage_date).days, nulos viram 0, retorna DataFrame.

compute_win_rates(df): filtra Won/Lost, calcula win_rate por sector e por sales_agent como Won/(Won+Lost), retorna dois dicts.

score_deal(row, sector_wr, agent_wr, max_value): retorna (score, tier, breakdown) com estas 6 features:
- deal_stage: Engaging=30, Prospecting=15, outros=0
- days_in_pipeline: ≤30=20, ≤60=15, ≤90=8, ≤180=3, resto=0
- close_value: round((valor/max_value)*20)
- win rate setor: round(sector_wr.get(sector,0.5)*15)
- win rate vendedor: round(agent_wr.get(agent,0.5)*10)
- tamanho conta: rev>10M ou emp>500=5, rev>1M ou emp>100=3, resto=1
Tier: A=70+, B=40-69, C=0-39. breakdown com labels legíveis em português.

score_all(df): filtra só deals abertos, chama compute_win_rates uma vez, aplica score_deal em cada linha, retorna DataFrame com score/tier/breakdown ordenado por score desc.

---

ai_insights.py com função get_recommendation(row, breakdown, api_key): chama claude-sonnet-4-20250514, prompt inclui todos os campos do deal + breakdown com pontuação por feature + win rates reais calculados do dataset, pede retorno SOMENTE JSON sem markdown com campos urgency("alta"|"media"|"baixa"), main_risk(1 frase com dado concreto do deal), next_action(ação específica para hoje ou amanhã), why_score(1 frase conectando os dados ao score). Remove ```json``` antes de parsear. try/except com fallback em todos os campos. Nunca lança exceção.

---

app.py:

Setup: page_config title="Lead Scorer" layout="wide" page_icon="📊", @st.cache_data(ttl=3600) no carregamento.

Sidebar: API key Anthropic(password) com instrução "Cole sua chave para análise com IA", filtros Vendedor/Manager/Região com opção "Todos", multiselect Tier default=[A,B], botão "Atualizar scores" limpa cache.

Header: 4 métricas — total deals visíveis, count Tier A, valor total Tier A em R$, score médio.

Aba 1 "Pipeline": tabela com account/deal_stage/sales_agent/regional_office/close_value/days_in_pipeline/score/tier, cores A=#d4edda B=#fff3cd C=#f8d7da, máximo 100 linhas, caption explicando as 6 features e seus pesos.

Aba 2 "Análise IA": selectbox top 20 deals da view filtrada. Col esquerda(1/3): métricas score+tier, gráfico barras horizontal plotly do breakdown com labels legíveis. Col direita(2/3): com API key mostra botão "Analisar com IA"→spinner→st.info(main_risk) +st.success(next_action)+st.caption(why_score); sem API key mostra st.warning; sempre mostra tabela com todos os campos do deal.

```
Aba 3 "Gestor": barras agrupadas por vendedor Tier A(verde)
+B(amarelo) ordenado por A desc, tabela por manager com vendedores/
deals_tier_a/deals_tier_b/valor_pipeline, scatter x=days_in_pipeline
y=score cor=tier hover=account+sales_agent.

Rodapé: caption com timestamp e explicação das 6 features com seus
pesos.

---

Regras:
- Leia os CSVs primeiro — use campos e valores exatos
- App funciona 100% sem API key
- Só plotly para gráficos
- Valores monetários em R$ com separador de milhar
- Após criar os arquivos: pip install -r requirements.txt e teste com
streamlit run app.py --server.headless true
- Ao finalizar informe: valores exatos de deal_stage encontrados,
qualquer ajuste feito por causa dos dados reais, e o comando exato
para rodar o app
```

vou jogar esse prompt lá agora, vai me orientando aqui
10:18

Perfeito. Cola o prompt lá e me manda o output que ele retornar — vou acompanhando cada etapa com você.
Enquanto ele processa, só um aviso: ele provavelmente vai pedir permissão para ler os arquivos e instalar dependências. Pode aprovar tudo.

eu seleciono a opçã worktree?
10:19

Não — deixa em **Local**. Worktree é para casos avançados de git que não precisamos agora.

ele me entregou isso:
Vou começar lendo todos os CSVs antes de escrever qualquer código.
Executado 12 comandos
All data analyzed. Key findings:

- deal_stage values: `Won`, `Engaging`, `Lost`, `Prospecting` (open = Engaging +

Prospecting)

- Revenue: in millions USD (range 4.54–11698)
- GTXPro in pipeline ≠ GTX Pro in products — 1480 rows need normalization
- close_value is NULL for all open deals → will use `sales_price` as proxy
- engage_dates are 2016–2017 → days_in_pipeline sempre >180 (score 0), será documentado
- Agora crio os 4 arquivos.
- Executado 3 comandos
- Creating solution files
- Criado 3 arquivos
- Now the main app:
- Criadoapp.py+314-0
- Agora instalo as dependências e testo:
- Executado 3 comandos
- Scoring funciona. Vou verificar a distribuição e testar o app:
- ExecutadoValidate score distribution and win rates
- Scoring válido. Agora testo o app no modo headless:
- Executado 2 comandos
- App inicializa sem erros. Vou verificar também se os imports do `ai_insights` funcionam:
- ExecutadoTest ai_insights import
- Tudo funcionando. Resumo final:
- Resultado
- Os 4 arquivos foram criados em `submissions/guilherme-pardin/solution/`:
- ArquivoStatusrequirements.txt ✅scoring.py ✅ai_insights.py ✅app.py ✅
- Valores exatos de `deal_stage` encontrados:
  - `Won` (4.238 deals) — histórico para win rate
  - `Lost` (2.473 deals) — histórico para win rate
  - `Engaging` (1.589 deals) → deals abertos, score de estágio = 30
  - `Prospecting` (500 deals) → deals abertos, score de estágio = 15
- Ajustes feitos com base nos dados reais:
  - `GTXPro` → `GTX Pro`: 1.480 linhas no pipeline usam "GTXPro" mas a tabela `products` tem "GTX Pro" — normalizado antes do merge.
  - `close_value` é NULL para todos os 2.089 deals abertos — substituído por `sales_price` do produto como valor esperado (`effective_value`).
- Revenue em milhões USD (confirmado via `metadata.csv`) — thresholds: `> 10` (>$10M) e `> 1` (>$1M).
  - `days_in_pipeline` sempre > 180 — dataset é de 2016–2017, então todos os deals abertos têm 3.400+ dias. A feature contribui 0 pts para todos (documentado). As outras 5 features ainda diferenciam bem.
- Distribuição de Tier: 7 deals Tier A (score ≥70), 2.082 Tier B — o Tier A só é atingível por deals `Engaging` com valor máximo e win rates altos. Comando para rodar:

```
cd submissions/guilherme-pardin/solution
streamlit run app.py
```

Mostrar mais
10:27

Excelente — ele fez um trabalho muito bom. Os ajustes que ele identificou sozinho são exatamente o tipo de coisa que impressiona no challenge:

- Detectou o `GTXPro` vs `GTX Pro` e normalizou
- Percebeu que `close_value` é NULL nos deals abertos e usou `sales_price` como proxy
- Documentou a limitação do `days_in_pipeline` (dataset de 2016-2017)

Agora vamos rodar o app. No terminal do VSCode:

```
cd "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge/submissions/guilherme-pardin/solution"
streamlit run app.py
```

Vai abrir automaticamente no browser em `localhost:8501`. Me manda um print do que aparecer.

de uma olhada:
http://localhost:8501/
tem como eu pedir alterações no front-end? como mudar essa cor na tabela pipeline pois não da para enxergar nada
10:31

Sim, pode pedir alterações direto no Claude Code. Manda essa mensagem para ele:

```
A tabela na Aba 1 "Pipeline" está com problema de legibilidade —
o texto não aparece nas células coloridas (A=#d4edda, B=#fff3cd,
C=#f8d7da).

Corrija a formatação da tabela para que:
1. O texto fique sempre visível — use cor de texto escura (#1a1a1a)
   em todas as células coloridas
2. A coluna close_value mostre valores formatados em R$ com milhar
3. A coluna score mostre a barra de progresso visual (st.progress
   ou barra colorida inline) além do número
4. A coluna tier mostre badge colorido: A=verde escuro,
   B=laranja escuro, C=vermelho escuro — texto branco

Após corrigir, teste novamente com streamlit run app.py --
server.headless true
```

Me manda o resultado quando ele terminar.

não consigo escrever no terminal e dar comando, como recuperar
10:34

No VSCode, clica no terminal que está rodando o Streamlit e aperta:

`Ctrl + C`
Isso para o servidor e libera o terminal para digitar novamente.

não esta liberando
10:35

Tenta `Cmd + C` (no Mac é Cmd, não Ctrl).
Se não funcionar, abre um novo terminal no VSCode:
Terminal → New Terminal
Vai abrir uma segunda aba de terminal limpa, sem precisar parar o Streamlit.

agora ele retornor, de uma olhada: http://localhost:8501/
10:37

Não consigo acessar URLs — só vejo imagens que você mandar aqui.
Tira um print da tela do browser e cola aqui que eu analiso e te digo o que pedir para o Claude Code corrigir.

não consigo tirar prints de todas as telas, mas esta rodando, só achei estranho o nome de algumas contas que esta 'nan', e a aba de pipeline, não gostei da cor de fundo da tabela, prefiro cores mais escuras para, algo mais. dark mode.
10:39

Manda isso para o Claude Code:

```
Corrija dois problemas no app.py:

1. VALORES NaN: Nas colunas account, sales_agent, regional_office e
   qualquer coluna de texto, substitua NaN por "—" (traço).
   Faça isso no load_and_merge() do scoring.py após o merge.

2. TABELA DARK MODE: Troque as cores de fundo da tabela na Aba 1
   por versões escuras com texto claro:
   - Tier A: fundo #1a3a2a, texto #4ade80 (verde escuro)
```

```
   - Tier B: fundo #3a2e0a, texto #fbbf24 (amarelo escuro)
   - Tier C: fundo #3a1a1a, texto #f87171 (vermelho escuro)
   - Linhas sem tier: fundo #1e1e1e, texto #e5e5e5

   Aplique também um CSS global via st.markdown para deixar
   toda a tabela com fundo escuro (#1e1e1e) e texto claro (#e5e5e5),
   sem bordas brancas visíveis.

Após corrigir teste com: streamlit run app.py --server.headless true
```

nan significa que não foi reconhecido o nome?
10:40

Exatamente. O `NaN` aparece quando o merge entre as tabelas não encontra correspondência — por exemplo, um `account` no `sales_pipeline.csv` que não existe exatamente igual no `accounts.csv` (pode ser diferença de espaço, maiúscula, acento).
O Claude Code já detectou um caso assim — o `GTXPro` vs `GTX Pro` nos produtos. Provavelmente tem algo similar nas contas ou vendedores.
Antes de só substituir por `"—"`, vale investigar a causa. Manda isso para o Claude Code:

```
Antes de corrigir os NaNs, investigue a causa:

1. Após fazer o merge no load_and_merge(), mostre quantas linhas
   ficaram com NaN em cada coluna (account, sector, sales_agent,
   manager, regional_office, product)

2. Para as colunas com mais NaN, mostre 5 exemplos dos valores
   do sales_pipeline que não casaram com as outras tabelas

3. Se for problema de normalização (espaços, maiúsculas, caracteres
   especiais) — corrija na origem antes do merge

4. Só use "—" como fallback para NaNs que realmente não têm
   correspondência nos dados
```

Assim a gente corrige o problema de verdade, não só esconde com um traço.

acho melho ao inves de traço colocar 'desconhecido' mesmo, ou vamos corrigir o que causa o erro para ele mostrar os nomes de todas as contas corretamente
10:42

Vamos corrigir a causa — mostrar "desconhecido" seria esconder um problema que tem solução. Se o Claude Code já normalizou o `GTXPro`, provavelmente tem mais

casos parecidos.
Manda isso para ele:

```
Investigue e corrija a causa raiz dos NaNs após o merge:

1. Antes do merge, normalize todas as chaves de join nas 4 tabelas:
   - strip() para remover espaços no início e fim
   - Padronize capitalização se necessário
   - Corrija variações conhecidas (ex: "GTXPro" → "GTX Pro")

2. Após aplicar as correções, faça o merge e mostre:
   - Quantas linhas ficaram com NaN em cada coluna
   - Quais valores do sales_pipeline ainda não casaram

3. Se ainda sobrar NaNs que não têm correspondência real nos dados,
   aí sim substitua por "Desconhecido"

4. Mostre o resultado final: total de deals abertos antes e depois,
   confirmando que nenhum deal foi perdido no merge
```

lembrando eu não mandei aquele outro prompt pra ele, então fale da correção do front da pipeline tambem
10:43

Manda isso para o Claude Code:

```
Faça duas correções no app:

## 1. CORREÇÃO DOS NaNs (scoring.py — load_and_merge())

Investigue e corrija a causa raiz dos NaNs após o merge:

- Antes do merge, normalize todas as chaves de join nas 4 tabelas:
  strip() para remover espaços, padronize capitalização, corrija
  variações conhecidas (ex: "GTXPro" → "GTX Pro")
- Após corrigir, mostre quantas linhas ainda ficaram com NaN
  e quais valores não casaram
- Para NaNs sem correspondência real nos dados, substitua por
  "Desconhecido"
- Confirme o total de deals abertos antes e depois do merge

## 2. TABELA DARK MODE (app.py — Aba 1 Pipeline)

Troque as cores da tabela por versões escuras com texto legível:
- Tier A: fundo #1a3a2a, texto #4ade80
- Tier B: fundo #3a2e0a, texto #fbbf24
- Tier C: fundo #3a1a1a, texto #f87171
- Linhas sem tier: fundo #1e1e1e, texto #e5e5e5

Aplique CSS global via st.markdown para deixar toda a tabela
com fundo escuro (#1e1e1e) e texto claro (#e5e5e5).
```

```
Após corrigir teste com: streamlit run app.py --server.headless true
```

ele me retornou isso:
Vou investigar os NaNs antes de escrever qualquer código.
Executado 3 comandos
Diagnóstico completo. Os NaNs têm duas causas:

1 1425 rows têm `account = NaN` genuíno no CSV (68% dos deals abertos) — cascateia em sector, revenue, employees
2 500 deals Prospecting têm `engage_date = NaN` (ainda não iniciaram engage)
3 GTXPro já estava tratado, mas strip não foi aplicado sistematicamente Agora corrijo os dois arquivos: Leu um arquivo, editado um arquivo Agora corrijo o app.py — dark mode na tabela e CSS global: Leu um arquivo, editado um arquivo Now find and replace the entire Tab 1 section's color constants and table rendering: Editadoapp.py+45-31 Agora testo ambas as correções: Executado 2 comandos Tudo sem erros. Resumo das duas correções:
4 NaNs corrigidos — `scoring.py`
5 Causa raiz diagnosticada:
6 `account` é genuinamente `NaN` para 1.425 linhas no CSV de origem (68% dos deals Engaging e Prospecting) — não é whitespace, são valores ausentes reais
7 Isso causava cascade de NaN em `sector`, `revenue`, `employees`, `office_location`
8 0 desajustes reais de chave após normalizar (exceto GTXPro já tratado)
9 Correções aplicadas:
10 `_normalize_str()` com `.strip()` aplicado em todos os 4 join keys antes do merge
11 `account` preserva NaN genuínos (não força strip em NaN)
12 Após o merge: todas as colunas string preenchidas com `"Desconhecido"` (1.425 accounts, 1.425 sectors)
13 `print()` no console mostra contagens e causa raiz a cada carregamento ColunaNaN antesDepois`account`1.4250 (→ "Desconhecido")`sector`1.4250 (→ "Desconhecido")`manager`00`regional_office`00
14 Dark mode — `app.py` Aba 1 ElementoCoresTier Afundo `#1a3a2a`, texto `#4ade80` (verde neon)Tier Bfundo `#3a2e0a`, texto `#fbbf24` (âmbar)Tier Cfundo `#3a1a1a`, texto `#f87171` (vermelho suave)Defaultfundo `#1e1e1e`, texto `#e5e5e5`Cabeçalhofundo `#111827`, texto `#d1d5db`Barra de scoretrilha `#374151`, preenchimento na cor do tierBadge tierfundo saturado por tier, texto na cor do tier CSS global injetado via `st.markdown(unsafe_allow_html=True)` para garantir que o wrapper `.ls-wrap` e a `.ls-table` herdem o fundo escuro.

Mostrar mais
10:49

Bom resultado. Ele descobriu algo importante — 1.425 deals genuinamente não têm `account` no CSV original, não é erro de normalização. Isso é uma limitação real dos dados que vale documentar na submissão.
Agora reinicia o app para ver as correções. No terminal do VSCode:

```
streamlit run app.py
```

Abre no browser e me manda um print das mudanças — especialmente a tabela da Aba 1 com o dark mode.

10:51

Ficou muito bom! Dark mode funcionando, cores legíveis, badges de tier, barra de score visual. Está com cara profissional.
Dois pontos que chamam atenção:

1. **"Dias no pipeline" está 3000+** — todos os deals abertos têm mais de 3.000 dias porque o dataset é de 2016-2017. O Claude Code já documentou isso, mas vale deixar uma nota visual no app.
2. **Muitos "Desconhecido"** — são os 1.425 deals sem account no CSV original. Limitação real dos dados.

Antes de continuar ajustes visuais, me diz: as outras duas abas estão funcionando? Clica em **Análise IA** e **Gestor** e me manda prints.

10:53

Está muito bom! As três abas funcionando, dark mode consistente, gráficos renderizando.
Algumas observações rápidas:
Aba Análise IA — perfeita. Score breakdown visual, tabela com todos os campos, aviso para inserir a chave. Exatamente o que o challenge pede.
Aba Gestor — funcional, mas o gráfico mostra quase só amarelo (Tier B) porque só 7 deals são Tier A no dataset inteiro. Isso é limitação dos dados, não do código.
Dois ajustes que valem fazer antes de submeter:
Manda isso para o Claude Code:

```
Dois ajustes finais:

1. Na Aba Gestor, o gráfico "Deals por Vendedor" está difícil de
```

```
   ler com tantos nomes no eixo X. Mude para mostrar só os top 15
   vendedores por total de deals, e rotacione os nomes em 45 graus.

2. O Score médio no header está mostrando "50.0" com uma casa
decimal.
   Arredonde para inteiro: exiba "50" sem o .0

Teste com: streamlit run app.py --server.headless true
```

Enquanto ele corrige, me diz: você já tem sua chave da API Anthropic para testar a Aba de Análise IA?

Não

10:55

Tudo bem — você pode submeter o challenge sem testar a IA agora. O scoring por regras já está funcionando e é o núcleo da solução.

Para conseguir a chave depois:

1. Acesse **platform.anthropic.com**
2. Crie uma conta
3. Vá em **API Keys → Create Key**
4. Cole na sidebar do app no campo "Chave Anthropic"

Por enquanto, foca nos ajustes finais e na submissão. Você mandou aquele último ajuste para o Claude Code?

sim ele me retornou isso: Duas linhas a mudar. Faço os dois edits:

Editado 2 arquivos, executado um comando

Feito. As duas mudanças foram cirúrgicas:

1. Score médio: `:.1f` → `round(...)}` — exibe `50` sem casa decimal
2. Gráfico Gestor: calcula coluna `Total = Tier A + Tier B`, filtra `.nlargest(15, "Total")`, rotação do eixo X de `-35°` para `-45°`, título atualizado para "Top 15 Vendedores"

10:59

Perfeito. Reinicia o app para ver as mudanças:

```
streamlit run app.py
```

Me manda print da aba Gestor para confirmar que ficou legível. Se estiver bom, partimos para a etapa final — montar o README de submissão e fazer o commit.

11:00

Ficou muito melhor — legível, top 15, tabela por manager com valor de pipeline. Está com cara de produto real.
O app está pronto. Agora partimos para a submissão.
Próximos passos:

1. Montar o README.md da submissão
2. Fazer commit de tudo
3. Push para o GitHub
4. Abrir o Pull Request